Коллоквиум по C++ ПИ 1 курс весна

github.com/int28t/hse-se-lecture-notes

Содержание

1 Вокруг языка C++ **8**

1.1 Какие стандарты языка существуют? Каков цикл выпуска новых стандартов? 8

1.2 Что такое The Zero Overhead Principle? Привести пример появления этого принципа в коде. 8

1.3 Какие основные директивы языка CMake/файлов проекта? Для чего нужна директива `add_executable`? Каковы её параметры? 9

1.4 Что такое интерфейс и что такое реализация? Что из них можно отнести к идеальному, а что — к материальному? 10

1.5 Привести конкретные примеры разделения на интерфейс и реализацию в языке C++ (модули, типы и т.д.). 10

1.6 Для чего используются заголовочные файлы в C++? Что в них обычно помещают? Что с ними обычно делают и что с ними обычно делать нельзя? Какие расширения встречаются у заголовочных файлов? Какая связь между заголовочными файлами и принципом интерфейса-реализации? 11

1.7 Для чего используются модули трансляции в C++? Что в них обычно помещают? Что с ними обычно делают и что с ними обычно делать нельзя? Какие расширения встречаются у модулей трансляции? Какая связь между модулями трансляции и принципом интерфейса-реализации? 12

2 Основы C++ **14**

2.1 Каково назначение препроцессора языка C(++)? С какими сущностями он манипулирует? Имеет ли для него принципиальное значение синтаксис и семантика языка C++? 14

2.2 Как на примере директивы `assert` препроцессор меняет код программы для конфигураций debug и release? 14

2.3 Что содержит и что не содержит листинг (результат работы препроцессора)? 15

2.4 Что делают директивы условного включения `#if`, `#ifdef`, `#ifndef`, `#else`, `#elif`, `#endif`? 15

2.5 Как можно конфигурировать текст исходного кода программы условными директивами? 16

2.6 Для чего используется директива `#include`? Чем отличаются системные и пользовательские заголовки? Каков порядок включения стандартных и пользовательских заголовков? 17

2.7 В чём проблема повторного косвенного включения заголовочного файла? Привести пример. 17

2.8 Что такое header guard и какую проблему он предотвращает? Привести пример. 18

2.9 Можно ли использовать заголовочный файл без header guard? Какие ограничения это накладывает? 18

2.10 Как именуют header guard с привязкой к пути к файлу и имени проекта? Можно ли использовать `#pragma once`? . . . 19
2.11 Как включить в модуль системный заголовочный файл? Как включить заголовочный файл из проекта? . . . 19
2.12 Что обычно кладут в заголовочный файл — интерфейсную часть некоторого модуля `foo`? А что обычно не кладут — и почему? . . . 20
2.13 Чем в C++ описание чего-либо принципиально отличается от определения? Как наблюдать эффект описания и эффект определения? . . . 21

3 Базовые конструкции C++ 22
3.1 Алфавит языка C++. Простая программа C++, структура, основные действующие лица . . . 22
3.2 Ключевые слова. Основные ключевые слова . . . 22
3.3 Идентификаторы, правило составления идентификаторов. Влияние кодстайла на правило именования . . . 23
3.4 Инструкция (statement) как предложение языка. Классификация инструкций 23
3.5 Инструкции описания объекта данных и объекта действия (функции). `extern`. Прототип функции . . . 24
3.6 Инструкции определения и описания объекта данных . . . 25
3.7 Инициализация объектов. Конструкции `=`, `()`, `{}`, `={}` . . . 25
3.8 Блок инструкций. Scope . . . 26
3.9 Функция как именованный блок инструкций с заданным входом и выходом 27
3.10 Объекты и типы. Характеристики объектов и характеристик типов . . . 27
3.11 Представление объектов в оперативной памяти. `sizeof()` . . . 28
3.12 Операционная семантика типа. Абстрактный тип данных . . . 28
3.13 Классификация типов в C++: трехтаксонная и двухтаксонная . . . 29
3.14 Целочисленные типы данных: размеры, знаковость, диапазоны. `<cstdint>`, `<climits>`, `<limits>`, Little Endian . . . 30
3.15 Типы чисел с плавающей точкой: `float`, `double` . . . 30
3.16 Инициализация числовых объектов. Числовые литералы, narrowing, `=` и `{}` . 31
3.17 POD-типы и их особенность . . . 31
3.18 Выражения и операторы. Арность, приоритет, instruction-expression. Инициализация и присваивание . . . 32
3.19 Пример дерева разбора выражения . . . 32
3.20 Приоритет и ассоциативность операторов . . . 32
3.21 Логические выражения и control flow. `if`, `if` с инициализацией, `switch`, тернарный оператор . . . 33
3.22 Циклы: `while`, `for`, range-based for, `do..while`. `break`, `continue`, `return` . . . 34
3.23 Краткие ответы на контрольные вопросы . . . 35

4 Потоки и строки STL 36
4.1 Что подразумевают под моделью ввода-вывода консольного приложения (CLI-application)? Какие основные компоненты ввода-вывода задействуются для него? . . . 36
4.2 Что такое стандартный поток ввода и стандартный поток вывода? Какими объектами они представлены? Какая область видимости этих объектов? . . 36
4.3 Какой заголовочный файл необходимо подключить, чтобы получить доступ к стандартным потокам ввода/вывода? Где они описаны и где определены? 36
4.4 Какие типы данных у стандартных потоков ввода и вывода? Какая иерархия наследования для потоковых типов STL? . . . 37

4.5 Какие операции используются для ввода и вывода? Можно ли использовать оператор ввода для потока вывода и наоборот? Как они работают с пробелами? Что возвращают? . . . 37
4.6 Какие функции ввода и вывода можно использовать наравне с операторами? Как прочитать целиком строку, содержащую пробелы? . . . 38
4.7 Что такое состояние потока? Какие флаги его идентифицируют? Какие ошибки поправимы, а какие нет? Как сбросить ошибки? . . . 38
4.8 Что такое неявное приведение потока к bool? Как оно применяется в проверках вида «если поток»? . . . 39
4.9 Приведите идиоматические конструкции чтения до конца потока . . . 39
4.10 Как можно переназначить стандартные потоки ввода и вывода на другие устройства? Нужна ли перекомпиляция? . . . 40
4.11 Что такое кодирование символов? Как решается проблема больших алфавитов? Как устроен Unicode? . . . 40
4.12 8-битные кодировки: примеры, ограничения, ASCII . . . 41
4.13 Что такое `std::stringstream` и когда он нужен? . . . 41
4.14 Типы символов в C++ и символьные литералы . . . 42
4.15 UTF-8 как кодировка переменной длины . . . 42
4.16 Статические C-массивы, `sizeof`, decay . . . 42
4.17 C-строки и символ конца строки `'\0'` . . . 43
4.18 Класс `std::string`: назначение, инициализация, методы, внутреннее представление . . . 43
4.19 Как читать токен и строку с пробелами в `std::string`? . . . 44
4.20 Связь между `sizeof(std::string)` и длиной строки `s.length()` . . . 44
4.21 Mutable или immutable? Capacity и length . . . 45
4.22 Оптимизация коротких строк (SSO) . . . 45
4.23 Преобразования строк и чисел в стиле C++ . . . 45
4.24 Конкатенация строк . . . 45
4.25 `std::stringstream`: назначение, конкатенация, преобразование, тестирование 46
4.26 Краткие ответы на контрольные вопросы . . . 46

5 Функциональная декомпозиция, ссылки и константность 48
5.1 Как решать большую задачу так, чтобы решение было корректным, проверяемым, воспроизводимым, переиспользуемым и читаемым? . . . 48
5.2 Что такое функция с точки зрения языка C++? Как определить функцию или процедуру? Какие основные компоненты функции? . . . 48
5.3 Что такое неполный тип `void`? Можно ли породить объект такого типа? Что такое `void*`? . . . 49
5.4 Свободные функции и функции-члены. Методы классов и статические функции-члены . . . 50
5.5 Частые уточнения: `std::getline`, статическая функция-член и методы . . . 50
5.6 Что такое side-effect и почему он нежелателен для свободных функций? Как `static` вносит состояние? . . . 50
5.7 Как устроено определение функции? Что такое prototype, caller и callee? . . . 51
5.8 Список формальных параметров, заголовок и прототип . . . 52
5.9 Функция: интерфейсная часть и реализация. Как это связано со сборкой программы? . . . 52
5.10 Точки выхода из функции: `return` в практике . . . 52
5.11 Признаки хорошей функции и принципы функциональной декомпозиции . . . 53
5.12 Реентерабельность, параметризуемость, связность и тестируемость . . . 53

5.13 Принципы S.O.L.I.D. 54
5.14 Формальные и фактические параметры. Сигнатура и перегрузка функций . 54
5.15 Модели взаимодействия функций 55
5.16 CV-квалификаторы. `const` и `volatile` 55
5.17 Константный объект: что сохраняется и что запрещается 55
5.18 Ссылка как ещё одно имя для объекта 56
5.19 Создание ссылки, binding, перепривязка, отложенное привязывание 56
5.20 Ссылки в range-based for 56
5.21 Ссылки как параметры функций 57
5.22 Передача по значению и по ссылке. Правило по умолчанию 57
5.23 Константная ссылка 58
5.24 Что такое «большие» и «малые» объекты? Правило трёх указателей 58
5.25 Идиоматические правила передачи объектов в функции 59
5.26 Краткие ответы на контрольные вопросы 59

6 Стандартная библиотека: основные концепции и линейные контейнеры 60
6.1 Что такое UB (неопределённое поведение) в C++ и какую опасность оно представляет? Приведите примеры. 60
6.2 Как и с помощью инструкций `typedef` и `using` определяются псевдонимы типов? Для чего это применяется? 60
6.3 Какие основные компоненты входят в состав стандартной библиотеки? . . . 61
6.4 Что такое контейнеры, алгоритмы, итераторы? Какова связь между ними? . 61
6.5 Какая парадигма программирования описывает концепцию: один алгоритм — множество реализаций? 62
6.6 В каком стиле стандартные алгоритмы применяются к стандартным контейнерам: ООП или функциональном? Почему? 62
6.7 Чем схожи и чем различаются линейные и ассоциативные контейнеры? . . . 62
6.8 Чем схожи и чем различаются ассоциативные упорядоченные и неупорядоченные контейнеры? 63
6.9 `std::vector`: для чего применяется и в чём его назначение? 63
6.10 `std::vector`: доступ к элементам через `operator[]` и `at()` 64
6.11 `std::vector`: что возвращает `operator[]`? 64
6.12 `std::vector`: варианты инициализации 64
6.13 `std::vector`: добавление и удаление элементов. `push_back()`, `pop_back()`, `insert()`, `erase()` 64
6.14 `std::vector`: распределение памяти, `reserve()`, `size()`, `capacity()` 65
6.15 Что происходит при росте `vector`? Что такое переаллоцирование и как оно влияет на итераторы? 65
6.16 `std::vector`: тип элементов и шаблонный параметр 66
6.17 `std::vector`: ограничения на тип элементов 66
6.18 Можно ли создать `std::vector<std::istream>`? 66
6.19 Можно ли создать вектор векторов? Чем ограничена вложенность? 67
6.20 Как `vector` хранит элементы: по значению, по ссылке или через указатель? 67
6.21 `std::vector` и `std::array`: сходства и различия 67
6.22 `std::array` и C-массивы: сходства и различия 68

7 Стандартная библиотека: итераторы 69
7.1 С какой целью функция, получающая параметр по ссылке, может возвращать тот же параметр тоже по ссылке (pass-through)? 69

7.2 Как условно объекты подразделяют на «большие» и «небольшие»? Какой критерий для этого применяется? . . . 69
7.3 Какие варианты последовательного перебора элементов линейного контейнера существуют? . . . 69
7.4 Для чего необходим механизм косвенной адресации, основанный на итераторах? 70
7.5 Сколько итераторов необходимо для представления одного элемента, двух элементов, n элементов? . . . 70
7.6 Как с помощью итераторов представляется range (диапазон) объектов? . . . 71
7.7 Что общего и что различается у итераторов `it1` и `it2` в диапазоне `[it1, it2)`? . . . 71
7.8 Допустимо ли разыменование `it1` и `it2` в диапазоне `[it1, it2)`? . . . 71
7.9 Какие операции применимы к подавляющему большинству итераторов? Как умеют «двигаться» итераторы? . . . 72
7.10 Чем различаются свободная функция `std::begin(c)` и метод `c.begin()`? . . 72
7.11 Какие типы итераторов встроены в стандартные контейнеры? Для чего несколько разновидностей? . . . 72
7.12 Какие категории итераторов выделяли до стандарта C++20? . . . 73
7.13 Какие типы итераторов использует `std::vector`? А `std::list`? . . . 73
7.14 Как итерировать от первого к «запоследнему» элементу и обратно? Какие операции при этом используются? . . . 74
7.15 Движение итераторов: `++it`, `it++`, `it += n`, `it + n` . . . 74
7.16 Как сравнивать итераторы при переборе коллекции: `it != c.end()` или `it < c.end()`? . . . 74
7.17 Можно ли копировать итератор `c.begin()`? Можно ли его разыменовать? Можно ли двигать вперёд? . . . 75
7.18 Можно ли копировать итератор `c.end()`? Можно ли его разыменовать? Можно ли двигать назад? . . . 75
7.19 Какие операции допустимы к итератору на валидный элемент и к итератору *past-the-end*? . . . 75
7.20 Какой тип итератора следует выбирать при разработке нового алгоритма: минимально достаточный или максимально сильный? . . . 75
7.21 Что происходит с итератором после удаления элемента из коллекции по позиции этим итератором? . . . 76
7.22 Когда итераторы остаются валидными, а когда нет? . . . 76
7.23 Краткие ответы на контрольные вопросы . . . 76

8 Косвенная адресация **78**
8.1 Что такое косвенная адресация и для чего она применяется? . . . 78
8.2 Какие механизмы языка C++ и компоненты стандартной библиотеки вовлекаются при косвенной адресации? . . . 78
8.3 Какое количество объектов как правило задействовано при косвенной адресации: при использовании указателя? ссылки? итератора? . . . 78
8.4 Косвенная адресация и константность. Один объект — две разновидности константности. Два объекта — четыре разновидности константности . . . 79
8.5 Указатели: «левоконстантный», «правоконстантный» и «дваждыконстантный» указатель . . . 79
8.6 Ссылки: константность и неконстантность . . . 80
8.7 Итераторы: `const_iterator`, `const iterator` и `const const_iterator` . . . 80
8.8 Почему для «левоконстантного» итератора требуется отдельный тип `const_iterator`? 80
8.9 Краткие ответы на контрольные вопросы . . . 81

9 Класс std::string_view **82**
9.1 Ключевое слово auto: назначение, правила применения, ограничения 82
9.2 Строковые литералы, глобальная доступность и слияние одинаковых литералов 82
9.3 std::string_view как разновидность косвенной адресации 83
9.4 Какие объекты нужны для представления Null-terminated строки? 83
9.5 Какие объекты нужны для представления суффикса, префикса и инфикса Null-terminated строки? 83
9.6 Два указателя против указателя + длина 84
9.7 Передача и возврат std::string_view из функций 84
9.8 Конструирование и копирование std::string_view. Почему это невладеющий тип? 85
9.9 Ограничения std::string_view как невладеющего типа 85
9.10 Ограничение std::string_view при работе со стандартной библиотекой . . 86
9.11 Ключевые операции над строками с помощью std::string_view 86
9.12 Что происходит, если модифицировать подстроку через std::string_view? 87
9.13 Краткие ответы на контрольные вопросы 87

10 Ассоциативные контейнеры std::set и std::map **88**
10.1 Какие основные характеристики имеют ассоциативные контейнеры? 88
10.2 Что такое std::set? Каковы основные операции и ограничения? 88
10.3 Какие требования накладываются на тип элементов std::set и почему? . . 89
10.4 Как итерироваться по std::set? К какой категории относятся итераторы? . 89
10.5 Как искать элемент в std::set? Что возвращает find()? 90
10.6 Как работают insert() и erase() в std::set? Почему их сложность связана с find()? 90
10.7 Как обычно реализован std::set? 90
10.8 Что такое std::pair? Почему он нужен? 91
10.9 Что такое std::map? Чем он отличается от std::set<std::pair<K,V»? . . . 91
10.10 Как std::map моделирует ассоциативный массив? 92
10.11 Как работает operator[] у std::map? Почему важна неконстантная перегрузка? 92
10.12 Как итерироваться по std::map? Что получается при разыменовании итератора? 92
10.13 Что происходит при вставке элемента в std::map, если ключ уже существует? 93
10.14 Как правильно искать элемент в std::map? 93
10.15 Чем отличаются std::multiset и std::multimap от std::set и std::map? . 93
10.16 Чем упорядоченные ассоциативные контейнеры отличаются от неупорядоченных? 94
10.17 Какая типичная внутренняя реализация у std::map и какова асимптотика операций? 94

11 Ассоциативные контейнеры, компараторы, сравнение и сортировка **95**
11.1 Каково назначение параметра Compare в шаблонах std::set<Key, Compare> и std::map<Key, Value, Compare>? 95
11.2 Какая типичная реализация у std::set и std::map? Почему при поиске используется сравнение? 95
11.3 Какие стандартные компараторы используются по умолчанию? 95
11.4 Как задаётся пользовательский компаратор? 96
11.5 Каковы способы реализации пользовательского компаратора? 96
11.6 Чем пользовательский компаратор отличается от перегрузки operator<? . . 97

11.7 Как устроена «автоматическая сортировка» ассоциативных контейнеров? . . 97
11.8 Как «сортируют» элементы непустого `std::set`? 97
11.9 Как «сортируют» элементы непустого `std::vector`? 98
11.10 Какие требования накладываются на линейный контейнер, чтобы его можно было сортировать `std::sort`? 98
11.11 Можно ли отсортировать поддиапазон `std::vector`? 98
11.12 Можно ли упорядочить только часть `std::set` или `std::map`? 99
11.13 Можно ли использовать один и тот же компаратор для `std::set`, `std::map` и `std::vector`? 99
11.14 Что такое неупорядоченные ассоциативные контейнеры? Как они устроены? 99
11.15 Когда неупорядоченные контейнеры полезнее упорядоченных? 100

1 Вокруг языка C++

1.1 Какие стандарты языка существуют? Каков цикл выпуска новых стандартов?

Язык C++ стандартизуется Международной организацией по стандартизации (ISO). Каждый стандарт фиксирует язык и стандартную библиотеку на некотором этапе развития.

Основные стандарты C++

Исторически приняты следующие крупные ревизии языка:

- **C++98** — первый международный стандарт;
- **C++03** — небольшое исправление и уточнение C++98;
- **C++11** — большой шаг с *auto*, *lambda*, *move semantics*, *range-based for*;
- **C++14** — доработка C++11;
- **C++17** — развитие языка и библиотеки;
- **C++20** — крупное обновление: *concepts*, *modules*, *ranges*;
- **C++23** — очередная ревизия с улучшениями языка и библиотеки;
- **C++26** — следующий стандарт, который на момент обсуждения находится в стадии разработки как draft.

Примечание

В разговорной практике часто говорят не только о *standard*, но и о *draft* — рабочем проекте стандарта. Именно по draft обычно ориентируются implementaions компиляторов, когда реализуют новые возможности раньше окончательной публикации стандарта.

После C++11 стандарты выпускаются примерно раз в три года: идея состоит в том, чтобы язык развивался регулярно, но без резких и слишком частых изменений.

1.2 Что такое The Zero Overhead Principle? Привести пример появления этого принципа в коде.

The Zero Overhead Principle

Zero Overhead Principle — принцип нулевых накладных расходов: если абстракция в C++ не используется, то она не должна ухудшать производительность по сравнению с низкоуровневым кодом; если используется, её цена должна быть минимальной и, по возможности, определяться на этапе компиляции.

Идея языка состоит в том, чтобы давать высокоуровневые абстракции, но не заставлять платить за них лишнее время выполнения, память или дополнительные косвенные вызовы.

Пример

Классический пример — шаблоны и ‘inline‘-функции:

```
template <class T>
T max_value(const T& a, const T& b) {
    return (a < b) ? b : a;
}
```

Здесь абстракция *generic programming* устраняет необходимость писать отдельную функцию для каждого типа. После инстанцирования шаблона компилятор получает конкретный код для ‘int‘, ‘double‘ и т.д., и при оптимизации он часто совпадает по стоимости с ручной реализацией.

Ещё один важный пример — RAII и умные указатели. Например, `std::unique_ptr` даёт безопасное управление ресурсом, но в типичном случае не добавляет дорогой динамической диспетчеризации.

1.3 Какие основные директивы языка CMake/файлов проекта? Для чего нужна директива `add_executable`? Каковы её параметры?

CMake — это не компилятор, а система генерации build system. В файле `CMakeLists.txt` описывают, что именно нужно собрать.

Типичные команды CMake

На практике чаще всего встречаются:

- `cmake_min_required(...)` — минимальная версия CMake;
- `project(...)` — имя проекта и языки;
- `add_executable(...)` — создание исполняемого target;
- `add_library(...)` — создание библиотеки;
- `target_sources(...)` — привязка исходников к target;
- `target_link_libraries(...)` — линковка с библиотеками;
- `target_include_directories(...)` — пути к заголовкам;
- `set(...)` — установка переменных и свойств.

`add_executable`

Команда `add_executable` создаёт target типа *executable*:

```
add_executable(<name> [WIN32] [MACOSX_BUNDLE] [EXCLUDE_FROM_ALL] <sources>...)
```

Её смысл:

- `<name>` — логическое имя цели сборки;
- `<sources>` — список исходных файлов;

- `WIN32` — пометка для Windows GUI-приложений;
- `MACOSX_BUNDLE` — сборка в виде bundle на macOS;
- `EXCLUDE_FROM_ALL` — не включать цель в стандартную сборку по умолчанию.

Пример

```
cmake_minimum_required(VERSION 3.20)
project(hello_cxx)

add_executable(hello main.cpp util.cpp)
```

Здесь CMake создаёт исполняемый файл `hello` из двух исходников.

1.4 Что такое интерфейс и что такое реализация? Что из них можно отнести к идеальному, а что — к материальному?

Интерфейс

Interface — это то, *что* объект, модуль или функция умеют делать и как с ними можно взаимодействовать. Интерфейс описывает контракт: сигнатуры функций, публичные типы, ограничения и обещания.

Реализация

Implementation — это то, *как* именно этот контракт выполняется внутри. Реализация содержит алгоритмы, данные, скрытые детали и технические решения.

Интерфейс обычно относят к **идеальному**: он задаёт абстракцию, независимую от конкретного кода. Реализацию относят к **материальному**: это уже конкретный текст программы, который компилируется, линкуется и исполняется.

Примечание

Хорошая архитектура старается минимизировать зависимость пользователя от реализации. Пользователь должен знать интерфейс, но не обязан знать внутреннее устройство.

1.5 Привести конкретные примеры разделения на интерфейс и реализацию в языке C++ (модули, типы и т.д.).

Разделение на интерфейс и реализацию в C++ встречается на нескольких уровнях.

- **Функции**: в `.h` объявление, в `.cpp` определение.
- **Классы**: в заголовке описывают публичные методы и поля, а в `.cpp` — их тела.
- **Абстрактные классы**: интерфейс задают pure virtual methods, а реализацию дают в наследниках.
- **Модули C++20**: *module interface unit* экспортирует интерфейс, а *module implementation unit* содержит скрытую реализацию.

- **PImpl idiom**: в заголовке виден только класс-обёртка, а настоящие поля спрятаны в реализации.

Пример

```
// interface: shape.h
struct Shape {
    virtual double area() const = 0;
    virtual ~Shape() = default;
};

// implementation: circle.cpp
struct Circle : Shape {
    double r;
    double area() const override { return 3.14159 * r * r; }
};
```

Здесь `Shape` — интерфейс, а `Circle` — конкретная реализация.

1.6 Для чего используются заголовочные файлы в C++? Что в них обычно помещают? Что с ними обычно делают и что с ними обычно делать нельзя? Какие расширения встречаются у заголовочных файлов? Какая связь между заголовочными файлами и принципом интерфейса-реализации?

Заголовочный файл

Header file — файл, который обычно подключают через `#include` и который предназначен для распространения интерфейса между несколькими *translation units*.

Обычно в заголовках размещают:

- объявления функций;
- определения классов и структур;
- объявления типов, alias-ов, enum-ов;
- `constexpr`, `inline`, шаблоны;
- константы и объявление внешних объектов, если они действительно должны быть общими.

Заголовок обычно:

- **подключают** в `.cpp`-файлы;
- **не компилируют** как отдельную единицу;
- **защищают** include guard-ами или `#pragma once`.

Что обычно **нельзя** делать в заголовке без особой необходимости:

- размещать обычные определения функций и переменных, если это приведёт к множественным определениям;
- включать тяжёлые зависимости без нужды;
- создавать циклические зависимости между заголовками;
- путать интерфейс с внутренними деталями реализации.

Типичные расширения: `.h`, `.hpp`, `.hh`, `.hxx`, иногда `.inl` для inline-частей.

Примечание

Связь с принципом interface/implementation здесь прямая: заголовок — это, как правило, место для интерфейса, а `.cpp`-файл — место для реализации.

1.7 Для чего используются модули трансляции в C++? Что в них обычно помещают? Что с ними обычно делают и что с ними обычно делать нельзя? Какие расширения встречаются у модулей трансляции? Какая связь между модулями трансляции и принципом интерфейса-реализации?

Модульная трансляционная единица

Module translation unit — это translation unit, в котором есть `module declaration`. Такая единица может быть *module interface unit*, *module implementation unit* или *partition*.

Модули в C++20+ введены для более строгого и явного разделения интерфейса и реализации, чем в системе заголовков. Они уменьшают зависимость от текстового включения и помогают ускорять сборку.

Обычно в **module interface unit** помещают:

- `export module ...;`
- экспортируемые объявления классов, функций, шаблонов;
- публичную часть API.

Обычно в **module implementation unit** помещают:

- неэкспортируемые определения;
- скрытые helper-функции;
- детали реализации, не нужные пользователю модуля.

Что с модулями обычно **делают**:

- импортируют через `import`;
- экспортируют только нужную часть API;
- разделяют публичное и скрытое содержимое более строго, чем в headers.

Что с ними обычно **не делают**:

- не используют как обычный заголовок с бесконтрольным текстовым `#include`;
- не дублируют экспорт без необходимости;
- не смешивают интерфейс и реализацию хаотично.

Типичные расширения: `.ixx`, `.cppm`, `.mpp`, иногда обычный `.cpp`.

Примечание

Модули лучше выражают принцип interface/implementation: интерфейс задаётся через `export`, а реализация скрывается внутри модуля. В этом смысле модуль — более современный и формальный механизм, чем заголовочный файл.

2 Основы C++

2.1 Каково назначение препроцессора языка C(++)? С какими сущностями он манипулирует? Имеет ли для него принципиальное значение синтаксис и семантика языка C++?

Препроцессор

Preprocessor — это этап обработки исходного текста *до* компиляции. Он выполняет текстовые преобразования: подключает файлы, разворачивает макросы, включает или исключает фрагменты кода по условиям.

Назначение препроцессора:

- подключение заголовков через `#include`;
- условная компиляция через `#if`, `#ifdef`, `#ifndef` и т.д.;
- определение и подстановка макросов через `#define`;
- организация конфигурации кода под разные платформы, режимы сборки и окружения.

Препроцессор работает не с объектами языка C++ как таковыми, а с **текстом исходника**. Поэтому для него принципиально важны строки, токены и директивы, но *не* семантика C++ в полном смысле: он не понимает типы, классы, перегрузку функций и другие правила компилятора.

Примечание

Препроцессор можно рассматривать как текстовый механизм, а не как часть собственно языка C++. Именно поэтому ошибки препроцессора часто выглядят грубо и локально, а не как «умные» сообщения о типах.

2.2 Как на примере директивы `assert` препроцессор меняет код программы для конфигураций debug и release?

Директива `#include <cassert>` подключает проверку `assert`. Её поведение зависит от того, определён ли макрос `NDEBUG`.

Debug и release

- **debug** — режим отладки: проверки обычно включены;
- **release** — режим поставки: проверки обычно выключены ради скорости.

Пример

```
#include <cassert>

int f(int x) {
    assert(x != 0);
    return 100 / x;
}
```

Если `NDEBUG` *не* определён, то `assert(x != 0)` работает как runtime-check: при ложном условии программа аварийно завершится, обычно с диагностикой. Если `NDEBUG` определён, то макрос `assert` разворачивается в пустую конструкцию вида `((void)0)` и не влияет на runtime.

Примечание

Это хороший пример принципа *zero overhead*: проверки удобны в debug, но не должны накладывать стоимость в release.

2.3 Что содержит и что не содержит листинг (результат работы препроцессора)?

Листинг

Listing — это текст, полученный после работы препроцессора, то есть уже «развёрнутый» исходник.

Обычно листинг содержит:

- подставленные заголовочные файлы;
- раскрытые макросы;
- результат условной компиляции;
- служебные директивы вида `#line` или аналогичные, помогающие компилятору сопоставлять строки с исходными файлами.

Обычно листинг *не содержит*:

- самих директив `#define`, `#include` в их исходном виде;
- комментариев как самостоятельного материала;
- логики компиляции по типам и семантике C++.

Примечание

Если посмотреть на листинг, можно увидеть, что препроцессор делает не «умное преобразование программы», а именно текстовую подстановку и фильтрацию фрагментов.

2.4 Что делают директивы условного включения `#if`, `#ifdef`, `#ifndef`, `#else`, `#elif`, `#endif`?

Эти директивы позволяют включать или исключать фрагменты исходного текста в зависимости от условий препроцессора.

Условная компиляция

Conditional compilation — это механизм, при котором разные части кода присутствуют или отсутствуют в итоговом translation unit в зависимости от макросов и выражений препроцессора.

Смысл директив:

- `#if <expr>` — включает блок, если выражение истинно;
- `#ifdef X` — эквивалентно проверке «определён ли макрос `X`»;
- `#ifndef X` — эквивалентно проверке «не определён ли `X`»;
- `#else` — альтернативная ветка;
- `#elif` — цепочка альтернативных условий;
- `#endif` — завершение условного блока.

Пример

```
#ifdef _WIN32
    // Windows-specific code
#elif defined(__linux__)
    // Linux-specific code
#else
    // Other platform
#endif
```

2.5 Как можно конфигурировать текст исходного кода программы условными директивами?

Условные директивы применяют, когда один и тот же проект должен по-разному собираться на разных платформах или в разных режимах.

Пример

```
#ifdef DEBUG
    std::cerr << "Debug mode\n";
#endif

#ifdef _WIN32
    // Windows implementation
#else
    // POSIX implementation
#endif
```

Типичные случаи:

- платформенно-зависимый код (`Windows`, `Linux`, `macOS`);
- включение отладочных проверок;
- включение экспериментальных возможностей;
- защита от неподдерживаемой конфигурации.

2.6 Для чего используется директива `#include`? Чем отличаются системные и пользовательские заголовки? Каков порядок включения стандартных и пользовательских заголовков?

Директива `#include` вставляет содержимое другого файла в текущий translation unit на этапе препроцессинга.

Системный и пользовательский include

- **Системный (библиотечный) заголовок**: подключается как `#include <...>`;
- **Пользовательский (проектный) заголовок**: подключается как `#include "..."`;

Разница в синтаксисе отражает намерение:

- `<...>` — искать в системных путях и путях, заданных сборкой;
- `"..."` — сначала искать рядом с текущим файлом, затем в системных путях.

Стилевое соглашение обычно такое:

1. сначала стандартные заголовки;
2. затем пустая строка;
3. потом проектные заголовки.

Примечание

Такой порядок делает зависимости более читаемыми и помогает быстро увидеть, где заканчивается стандартная библиотека и начинается код проекта.

2.7 В чём проблема повторного косвенного включения заголовочного файла? Привести пример.

Повторное косвенное включение

Повторное косвенное включение возникает, когда один и тот же заголовок попадает в translation unit несколько раз не напрямую, а через цепочку других заголовков.

Пример

Пусть:

```
// a.h
#include "common.h"

// b.h
#include "common.h"

// main.cpp
#include "a.h"
#include "b.h"
```

Тогда `common.h` будет включён дважды, если не защитить его от повторного включе-

ния.

Проблема заключается в том, что без специальных мер повторное включение может привести к:

- множественным определениям;
- повторному объявлению типов и функций;
- конфликтам имён;
- ошибкам компиляции.

2.8 Что такое header guard и какую проблему он предотвращает? Привести пример.

Header guard

Header guard — это классическая защита заголовочного файла от повторного включения.

Классическая реализация:

```
#ifndef PROJECT_FOO_BAR_H
#define PROJECT_FOO_BAR_H

// contents of foo/bar.h

#endif // PROJECT_FOO_BAR_H
```

Смысл: при первом включении макрос ещё не определён, поэтому содержимое заголовка попадает в translation unit, а затем макрос определяется. При повторном включении содержимое пропускается.

Пример

```
#ifndef MYPROJECT_UTILS_VECTOR_H
#define MYPROJECT_UTILS_VECTOR_H

struct Vec {
    double x, y;
};

#endif
```

2.9 Можно ли использовать заголовочный файл без header guard? Какие ограничения это накладывает?

Технически можно, но только если вы **гарантируете**, что такой заголовок попадёт в каждый translation unit не более одного раза.

На практике это ограничение слишком жёсткое: при росте проекта один и тот же заголовок почти неизбежно начинает подключаться косвенно через разные файлы.

Если guard-ов (или `#pragma once`) нет, то возникают риски:

- повторные объявления и определения;
- ошибки компиляции из-за конфликтов;
- хрупкая зависимость от порядка `#include`;
- сложная поддержка и рефакторинг.

Примечание

Поэтому в обычном C++-коде заголовок без защиты от повторного включения считают плохой практикой. Исключения бывают только в узких технических сценариях, где повторное включение задумано специально.

2.10 Как именуют header guard с привязкой к пути к файлу и имени проекта? Можно ли использовать `#pragma once`?

Стилевое соглашение для header guard обычно такое:

- имя в `UPPER_CASE`;
- в него включают имя проекта;
- затем путь к файлу, нормализованный к символам `_`;
- часто добавляют суффикс `_H` или `_HPP`.

Пример

Если файл лежит как `project/include/math/vector.h`, то guard может выглядеть так:

```
PROJECT_INCLUDE_MATH_VECTOR_H
```

Вместо классического guard иногда используют:

```
#pragma once
```

Это короткий и удобный способ запретить повторное включение. На практике он широко поддерживается компиляторами, но классический header guard остаётся максимально переносимым и стандартизованным способом.

Примечание

Если в курсе требуется именно стиль с guard-ами, лучше уметь писать и понимать оба варианта.

2.11 Как включить в модуль системный заголовочный файл? Как включить заголовочный файл из проекта?

В модульной модели C++ есть несколько вариантов подключения.

Если речь идёт о **module interface unit** или **module implementation unit**, то стандартные заголовки обычно подключают либо через `#include`, либо через импорт стандартной библиотеки, если это поддерживается инструментами и стандартной библиотекой в данной среде.

Пример

```
module;

#include <vector>
#include "project/config.h"

export module foo;

export int f();
```

Здесь:

- `#include <vector>` — системный заголовок;
- `#include "project/config.h"` — заголовок проекта;
- строка `module;` означает *global module fragment*, где разрешены обычные include перед объявлением модуля.

2.12 Что обычно кладут в заголовочный файл — интерфейсную часть некоторого модуля `foo`? А что обычно не кладут — и почему?

В заголовок обычно кладут только **интерфейс**:

- объявления функций;
- определения классов и структур;
- объявления шаблонов;
- константные объявления и alias-ы;
- public API, которым пользуется внешний код.

Обычно не кладут:

- тяжёлую реализацию алгоритмов;
- внутренние helper-функции;
- подробности хранения данных;
- всё, что можно скрыть, не ломая API.

Причины:

1. уменьшить связанность кода;
2. сократить время компиляции;
3. скрыть детали реализации;
4. избежать повторного включения и проблем с множественными определениями.

Примечание

Хороший заголовок отвечает на вопрос «*что умеет модуль*», а не «*как он это делает*».

2.13 Чем в C++ описание чего-либо принципиально отличается от определения? Как наблюдать эффект описания и эффект определения?

Declaration

Declaration — это описание сущности для компилятора: мы сообщаем, что такая сущность существует и как с ней можно работать.

Definition

Definition — это определение сущности, то есть создание конкретного объекта, функции или другого элемента программы.

Формулировка в простом виде:

- **описать** — значит сообщить компилятору о существовании сущности;
- **определить** — значит реально создать сущность, то есть выделить для неё память или дать тело функции.

Пример

```
extern int x;   // declaration
int x = 5;      // definition
```

Здесь первая строка сообщает о переменной `x`, но не создаёт её. Вторая строка уже создаёт объект `x` в памяти.

Для функций:

```
int sum(int, int);      // declaration
int sum(int a, int b) { // definition
    return a + b;
}
```

Примечание

Эффект описания наблюдается в том, что компилятор начинает «знать имя» и тип сущности. Эффект определения наблюдается в том, что сущность получает реальное существование в программе: переменная начинает занимать память, а функция — иметь тело, которое можно вызвать и слинковать.

3 Базовые конструкции C++

3.1 Алфавит языка C++. Простая программа C++, структура, основные действующие лица

Алфавит языка

Алфавит C++ — это набор допустимых символов, из которых строится исходный текст программы: латинские буквы, цифры, знаки препинания, пробелы, табуляции, символы переноса строки и специальные символы препроцессора.

В исходном тексте программы обычно выделяют следующие «действующие лица»:

- **препроцессор** — обрабатывает директивы `#include`, `#define`, `#if` и т.д.;
- **компилятор** — проверяет синтаксис и семантику, строит объектный код;
- **линкер** — собирает объектные файлы и библиотеки в исполняемую программу;
- **runtime-среда** — окружение, в котором программа реально выполняется.

По уровню видимости в базовой программе обычно встречаются:

- **namespace scope** (в т.ч. глобальная область);
- **block scope** внутри функций и управляющих конструкций.

Связь с инструментами сборки прямая: директивы обрабатывает препроцессор, синтаксис и типы — компилятор, связывание внешних определений — линкер.

Минимальная программа в C++ обычно содержит функцию `main`:

```
int main() {
    return 0;
}
```

Примечание

С точки зрения языка C++ это уже полноценная программа. Всё остальное — подключение библиотек, объявление типов, функции, классы — лишь расширяет её структуру.

3.2 Ключевые слова. Основные ключевые слова

Ключевое слово

Keyword — это зарезервированное слово языка C++, имеющее специальный синтаксический смысл и не допускающееся в качестве идентификатора.

К основным ключевым словам относятся:

- для типов и объявлений: `int`, `double`, `char`, `bool`, `class`, `struct`, `enum`, `void`;
- для управления потоком: `if`, `else`, `switch`, `case`, `for`, `while`, `do`, `break`, `continue`, `return`;
- для описания сущностей: `const`, `constexpr`, `static`, `extern`, `inline`, `virtual`, `namespace`;

- для управления памятью: `new`, `delete`;
- для исключений: `try`, `catch`, `throw`;
- для управления доступом: `public`, `private`, `protected`;

Примечание

Не все «служебные слова» языка являются ключевыми в строгом смысле, но для коллоквиума важно понимать, что они имеют фиксированное значение и не могут использоваться произвольно.

3.3 Идентификаторы, правило составления идентификаторов. Влияние кодстайла на правило именования

Идентификатор

Identifier — имя объекта, функции, типа, namespace, макроса и других сущностей программы.

Формально идентификатор в C++:

- состоит из букв, цифр и символа подчёркивания;
- не может начинаться с цифры;
- чувствителен к регистру;
- не должен совпадать с ключевыми словами.

Примеры:

- корректные: `value`, `myVar2`, `_tmp`;
- некорректные: `2value`, `my-var`, `class`.

С учётом code style обычно придерживаются такой схемы:

- объекты и функции — `camelCase`;
- типы — `PascalCase`;
- глобальные константы — `kPascalCase` или `UPPER_CASE`;
- макросы препроцессора — `UPPER_CASE`.

Примечание

Стандарт языка даёт лишь минимальные ограничения. Практика code style добавляет дополнительные соглашения: например, типы пишут в `PascalCase`, функции и переменные — в `camelCase`, константы — в `kPascalCase` или `UPPER_CASE`, а макросы — только `UPPER_CASE`.

3.4 Инструкция (statement) как предложение языка. Классификация инструкций

Инструкция

Statement — это законченная синтаксическая конструкция языка C++, выражающая действие или описание.

Обычно инструкции разделяют на несколько больших классов:

- **инструкция-описание** — объявляет сущность;
- **инструкция-определение** — создаёт сущность;
- **инструкция-выражение** — вычисляет выражение и завершается `;`;
- **инструкции управления потоком** — `if`, `switch`, `for`, `while`, `do`, `break`, `continue`, `return`;
- **блок инструкций** — составная инструкция в фигурных скобках.

Примечание

Не каждое законченное действие в тексте программы начинается с новой строки, и не каждая строка — это отдельная инструкция. Видимым признаком инструкции обычно служит точка с запятой или фигурные скобки, но есть исключения: например, блок и управляющие конструкции могут не оканчиваться `;`.

3.5 Инструкции описания объекта данных и объекта действия (функции). `extern`. Прототип функции

Описание

Declaration — это сообщение компилятору о существовании сущности и её свойствах без обязательного создания самой сущности.

Описание объекта данных

Это инструкция, которая сообщает компилятору о переменной или константе:

```
extern int x;      // declaration only

const double pi = 3.14159; // definition
```

Описание функции

Это инструкция, которая сообщает сигнатуру функции:

```
int sum(int a, int b);
```

Такая запись называется *prototype* или *function declaration*.

Ключевое слово `extern` обычно означает, что определение находится в другом месте:

```
extern int counter;
```

Примечание

`extern` особенно важно для глобальных объектов и связки между несколькими translation units.

3.6 Инструкции определения и описания объекта данных

Определение

Definition — это инструкция, которая создаёт сущность. Для объекта данных это обычно означает выделение памяти.

Пример

```
int a;          // definition + declaration
extern int b;   // declaration only
int c = 10;     // definition + declaration
```

Примечание

В C++ определение объекта данных почти всегда одновременно является и его описанием. Исключение — `extern`-объявление, которое только описывает объект.

Для функций действует та же идея:

- может быть только описание (прототип), а определение — в другом модуле трансляции;
- определение функции одновременно является и её описанием.

3.7 Инициализация объектов. Конструкции `=`, `()`, `{}`, `={}`

Инициализация

Initialization — задание начального значения объекту в момент его создания.

Основные формы:

- **copy-initialization**: `T x = value;`
- **direct-initialization**: `T x(value);`
- **list-initialization**: `T x{value};`
- **copy-list-initialization**: `T x = {value};`

Общее:

- все они могут задавать начальное значение;
- все относятся к созданию объекта.

Различия:

- `{}` лучше ловит narrowing;

- `()` может вызывать конструкторы и иногда попадает в эффект *most vexing parse*;
- `=` исторически выглядит как присваивание, но при объявлении это именно инициализация;
- `={}` сочетает синтаксис `=` и семантику списка инициализации.

Пример

```
int a = 5;
int b(5);
int c{5};
int d = {5};
```

Примечание

`{}` часто называют унифицированным инициализатором. Его обычно рекомендуют по умолчанию, когда важно получить единый стиль и защиту от сужающих преобразований.

3.8 Блок инструкций. Scope

Блок инструкций

Compound statement — это последовательность инструкций в фигурных скобках `{ }`.

Блок используется для:

- последовательного выполнения нескольких инструкций;
- создания локальной области видимости;
- группировки инструкций в теле `if`, `for`, `while`, функции и т.д.

Scope

Scope — область видимости, в пределах которой имя доступно без дополнительной квалификации.

Блок создаёт **local scope**: объявленные внутри него сущности живут и видны только до конца блока.

Пример

```
{
    int x = 1;
    int y = x + 2;
}
```

Такой блок можно использовать и без заголовка `if/for/while`: это удобно, чтобы ограничить время жизни временных объектов и не «протекать» именами дальше по коду.

3.9 Функция как именованный блок инструкций с заданным входом и выходом

Функция

Function — именованный блок инструкций с параметрами на входе и, как правило, возвращаемым значением на выходе.

Функция задаёт:

- имя;
- список параметров;
- возвращаемый тип;
- тело — блок инструкций;
- контракт вызова.

Пример

```
int sum(int a, int b) {
    return a + b;
}
```

3.10 Объекты и типы. Характеристики объектов и характеристик типов

Объект

Объект — это область памяти, в которой хранятся данные некоторого типа.

Тип

Тип — это описание множества значений и множества допустимых операций над ними.

Характеристики объекта:

- имя (может отсутствовать);
- тип;
- значение;
- адрес;
- размер;
- время жизни;
- область видимости имени.

Характеристики типа:

- множество допустимых значений;

- операции;
- представление в памяти;
- размер и выравнивание;
- правила инициализации, преобразований и сравнения.

В C++ объект без типа существовать не может: тип обязателен, потому что именно он задаёт размер, допустимые операции и правила интерпретации памяти.

Примечание

В C++ тип и объект тесно связаны, но это не одно и то же: тип определяет свойства, а объект является конкретной реализацией этих свойств в памяти.

3.11 Представление объектов в оперативной памяти. `sizeof()`

Footprint

Footprint объекта — это то, сколько памяти объект занимает и как эта память устроена.

На представление влияют:

- тип объекта;
- размер типа;
- выравнивание (*alignment*);
- способ кодирования чисел;
- наличие служебных полей у классов;
- оптимизации компилятора.

Оператор `sizeof` позволяет узнать размер объекта или типа:

```
sizeof(int)
sizeof x
```

- применим к типам и объектам;
- результат имеет тип `std::size_t`;
- возвращает размер в байтах.

Примечание

`sizeof` не вычисляет выражение как runtime-операция; обычно это compile-time информация.

3.12 Операционная семантика типа. Абстрактный тип данных

Операционная семантика

Операционная семантика типа — это набор операций, которые можно выполнять над объектами этого типа, и смысл этих операций.

Например, для `int` доступны сложение, вычитание, сравнение, инкремент и т.д.

ADT

Abstract Data Type — абстрактный тип данных, для которого важны интерфейс и набор операций, а не конкретная внутренняя реализация.

Примеры ADT:

- стек;
- очередь;
- множество;
- ассоциативный словарь.

3.13 Классификация типов в C++: трехтаксонная и двухтаксонная

Классическая **трёхтаксонная** классификация:

- **фундаментальные** типы;
- **производные** типы;
- **пользовательские** типы.

- **Фундаментальные**: `int`, `char`, `bool`, `float`, `double`, `void`, `nullptr_t`.
- **Производные**: указатели, ссылки, массивы, функции, указатели на члены.
- **Пользовательские**: `class`, `struct`, `enum`, `union`.

По cppreference часто используют **двухтаксонную** классификацию:

- **fundamental**;
- **compound (derived-like)**.

Типичные модификаторы для производных типов в учебной трёхтаксонной схеме:

- указатель: `T*`;
- ссылка: `T&`, `T&&`;
- массив: `T[N]`;
- функция: `R(Args...)`;
- указатель на член класса: `T C::*`.

Примечание

Для курса полезно уметь оперировать обеими схемами: трёхтаксонная удобнее для учебного объяснения, а двухтаксонная чаще встречается в документации.

3.14 Целочисленные типы данных: размеры, знаковость, диапазоны. `<cstdint>`, `<climits>`, `<limits>`, Little Endian

Целочисленные типы

К целочисленным типам относятся `char`, `short`, `int`, `long`, `long long` и их unsigned-версии.

- размеры зависят от платформы, кроме некоторых минимальных гарантий стандарта;
- знаковость определяется либо явно (`signed`/`unsigned`), либо по умолчанию;
- диапазон зависит от числа бит и способа кодирования.

Псевдонимы фиксированного размера из `<cstdint>`:

- `std::int8_t`, `std::uint8_t`;
- `std::int16_t`, `std::uint16_t`;
- `std::int32_t`, `std::uint32_t`;
- `std::int64_t`, `std::uint64_t`.

Диапазоны целых типов можно смотреть:

- в `<climits>` — через макросы вроде `INT_MIN`, `INT_MAX`;
- в `<limits>` — через `std::numeric_limits<T>`.

Примечание

Для хранения многобайтовых чисел важен порядок байт. На большинстве современных платформ используется *Little Endian*: младший байт идёт по меньшему адресу.

3.15 Типы чисел с плавающей точкой: `float`, `double`

Основные типы с плавающей точкой:

- `float`;
- `double`;
- `long double`.

Их представление обычно основано на стандарте IEEE 754 и включает:

- знак;
- порядок (exponent);

- мантиссу (fraction/significand).
- `float` обычно занимает 4 байта;
- `double` обычно 8 байт;
- `long double` зависит от платформы.

Такие типы позволяют представлять:

- обычные числа;
- очень большие и очень маленькие числа;
- специальные значения: `+inf`, `-inf`, `NaN`.

3.16 Инициализация числовых объектов. Числовые литералы, narrowing, `=` и `{}`

Числовой литерал может уточняться суффиксами:

- `1u` — `unsigned`;
- `1l` — `long`;
- `1ll` — `long long`;
- `1.0f` — `float`;
- `1.0l` — `long double`.

Narrowing

Narrowing — сужение типа, при котором значение может потерять часть информации, например при переводе `double` в `int`.

Сравнение:

- `int x = 3.14;` — может пройти с потерей дробной части;
- `int x{3.14};` — ошибка компиляции из-за narrowing.

Примечание

Именно поэтому список инициализации `{}` считается более безопасным.

3.17 POD-типы и их особенность

POD

POD (*Plain Old Data*) — исторический класс простых типов без сложного поведения. В современном C++ это понятие в значительной мере заменено более точными категориями вроде *trivial* и *standard-layout*.

Примеры учебно:

- `int`;
- `double`;
- простая `struct` без пользовательских конструкторов и виртуальных функций.

Примечание

Особенность таких типов — простота представления в памяти и предсказуемое копирование побайтно, если это допускается стандартом.

3.18 Выражения и операторы. Арность, приоритет, instruction-expression. Инициализация и присваивание

Выражение

Expression — конструкция языка, которая вычисляется и даёт значение или effect.

Оператор и операнд

Оператор — символ или ключевое слово, задающее операцию. Операнд — объект, над которым выполняется операция.

Выражение может состоять из одного операнда (например, `x`). Из одного оператора без операндов выражение составить нельзя.

- по арности операторы бывают унарные, бинарные и тернарные;
- приоритет определяет порядок вычисления;
- если порядок неочевиден, его задают скобками.

Instruction-expression

Инструкция-выражение — это выражение, завершённое `;`, например:

```
x = y + 1;
f();
```

Примечание

Инициализация и присваивание — не одно и то же. Инициализация происходит при создании объекта, а присваивание — после его создания.

3.19 Пример дерева разбора выражения

Для выражения

$$1 + 2 * 3$$

дерево разбора строится с учётом приоритета операторов: сначала связывается `2 * 3`, потом выполняется сложение с `1`. Скобки `(1 + 2) * 3` меняют дерево разбора и, как следствие, результат вычисления.

3.20 Приоритет и ассоциативность операторов

Приоритет оператора показывает, какие операции связываются раньше при разборе выражения. Если у нескольких операторов одинаковый приоритет, используется ассоциативность: слева направо или справа налево.

- приоритет определяет структуру дерева разбора;

- скобки всегда сильнее приоритета;
- ассоциативность нужна только тогда, когда приоритет одинаковый.

Примечание

Например, в `a - b - c` вычитание обычно левоассоциативно: выражение читается как `(a - b) - c`. У присваивания `=` ассоциативность обычно правая.

3.21 Логические выражения и control flow. `if`, `if` с инициализацией, `switch`, тернарный оператор

Control flow

Control flow — это управление порядком выполнения инструкций в программе.

Логические операции:

- `&&` — logical and;
- `||` — logical or;
- `!` — logical not;
- сравнения: `==`, `!=`, `<`, `<=`, `>` , `>=`.

`if`:

```
if (cond) {
    ...
} else {
    ...
}
```

`if` с инициализацией:

```
if (auto x = f(); x > 0) {
    ...
}
```

`switch` удобен для выбора по целочисленному или перечислимому значению:

```
switch (n) {
case 1:
    ...
    break;
default:
    ...
}
```

`break` в `case` обычно обязателен, иначе произойдёт проваливание в следующую ветку. Намеренное проваливание допустимо, но его надо явно отмечать (например, `[[fallthrough]];`) и использовать осознанно.

`switch` с инициализацией (C++17):

```
switch (int x = f(); x) {
case 1:
    ...
    break;
default:
    ...
}
```

Тернарный оператор:

```
cond ? expr1 : expr2
```

Он компактнее `if`, но удобен в основном для выбора значения, а не для сложной логики с несколькими инструкциями.

Ключевое отличие: тернарный оператор — это **выражение**. Его можно использовать там, где нужен результат выражения (инициализация, аргументы вызова, `return`), тогда как `if` — это инструкция.

3.22 Циклы: `while`, `for`, range-based for, `do..while`. `break`, `continue`, `return`

`while` используется, когда число итераций заранее неизвестно:

```
while (cond) {
    ...
}
```

`for` удобен для циклов со счётчиком:

```
for (int i = 0; i < n; ++i) {
    ...
}
```

Range-based for применяют для обхода коллекции:

```
for (const auto& x : container) {
    ...
}
```

`do..while` гарантирует хотя бы одну итерацию:

```
do {
    ...
} while (cond);
```

`break` завершает цикл или `switch`, `continue` переходит к следующей итерации, `return` завершает функцию.

Примечание

Range-based for можно переписать классическим `for` через итераторы, если коллекция поддерживает `begin()` и `end()`.

Эквивалентная форма:

```
for (auto it = container.begin(); it != container.end(); ++it) {
    const auto& x = *it;
    ...
}
```

Требования к перечисляемой коллекции: должна существовать пара границ диапазона (через `begin/end` или совместимый ADL-интерфейс), а элементы должны быть доступны через разыменование итератора.

3.23 Краткие ответы на контрольные вопросы

- **Что такое инструкция?** Законченная синтаксическая конструкция языка, обычно отделяемая `;` или оформленная блоком.
- **Что такое описание?** Сообщение компилятору о сущности без обязательного создания.
- **Что такое определение?** Создание сущности, например объекта или функции.
- **Может ли быть только описание без определения?** Да, например `extern int x;`.
- **Может ли быть только определение без отдельного описания?** Да, потому что определение обычно само по себе является и описанием.
- **Можно ли использовать блок без заголовка?** Да, как отдельный локальный scope.
- **Как получить named block?** Через функцию.
- **Есть ли объект без типа?** В C++ нет: объект всегда имеет тип.
- **Какая разница между `short` и `int`?** Помимо размера, `int` обычно является базовым типом, к которому выполняются integer promotions; `short` часто приводится к `int` в выражениях.
- **Какая разница между `char` и `int`?** `char` — отдельный тип для символов и байтов; он может быть signed или unsigned и участвует в специальных правилах преобразования.

4 Потоки и строки STL

4.1 Что подразумевают под моделью ввода-вывода консольного приложения (CLI-application)? Какие основные компоненты ввода-вывода задействуются для него?

CLI-application

CLI-application (*Command Line Interface application*) — это программа, которая взаимодействует с пользователем через текстовый ввод и вывод в консоли.

Для такой программы основными каналами являются:

- **standard input** (`stdin`) — стандартный ввод;
- **standard output** (`stdout`) — стандартный вывод;
- **standard error** (`stderr`) — стандартный поток ошибок.

Обычно пользователь вводит данные с клавиатуры, а программа печатает результат в консоль. С точки зрения библиотеки STL это выражается через потоки ввода и вывода.

4.2 Что такое стандартный поток ввода и стандартный поток вывода? Какими объектами они представлены? Какая область видимости этих объектов?

Стандартный поток ввода в C++ представлен объектом `std::cin`, а стандартный поток вывода — объектом `std::cout`.

Стандартные потоки

- `std::cin` — объект типа `std::istream`;
- `std::cout` — объект типа `std::ostream`.

Эти объекты находятся в namespace `std` и имеют глобальную область видимости в смысле времени жизни программы: они существуют всё время выполнения программы.

4.3 Какой заголовочный файл необходимо подключить, чтобы получить доступ к стандартным потокам ввода/вывода? Где они описаны и где определены?

Чтобы использовать `std::cin`, `std::cout`, `std::cerr`, обычно подключают:

```
#include <iostream>
```

Declaration и definition

Declaration сообщает компилятору о сущности, а *definition* создаёт её.

В случае стандартных потоков:

- в заголовке `<iostream>` они **объявлены**;
- их реальные объекты **определены** в стандартной библиотеке.

Это хороший пример различия между объявлением и определением: компилятор должен знать о существовании объекта, но сам объект создаётся библиотекой.

4.4 Какие типы данных у стандартных потоков ввода и вывода? Какая иерархия наследования для потоковых типов STL?

Базовые типы потоков:

- `std::istream` — входной поток;
- `std::ostream` — выходной поток;
- `std::iostream` — поток, совмещающий ввод и вывод.

Иерархия потоков

В STL потоковые классы образуют иерархию:

- `std::ios_base` — базовый уровень;
- `std::basic_ios<...>` — общая часть для ввода/вывода;
- `std::basic_istream<...>` и `std::basic_ostream<...>` — отдельные ветви;
- `std::basic_iostream<...>` — объединённый ввод-вывод;
- производные классы: `std::ifstream`, `std::ofstream`, `std::fstream`, `std::istringstream`, `std::ostringstream`, `std::stringstream`.

Примечание

Стандартная библиотека использует виртуальное наследование в этой иерархии, чтобы избежать проблемы «ромба» (*Deadly Diamond of Death*): общая базовая часть не дублируется при наследовании по двум ветвям.

4.5 Какие операции используются для ввода и вывода? Можно ли использовать оператор ввода для потока вывода и наоборот? Как они работают с пробелами? Что возвращают?

Для ввода используют оператор `»`, для вывода — оператор `«`.

- `std::cin » x;` читает данные из потока в переменную;
- `std::cout « x;` записывает данные в поток.

Примечание

Оператор `»` применяют к **входным** потокам, а `«` — к **выходным**. В типичном случае их нельзя «перепутать» без ошибок типов.

Для текстовых данных:

- `»` читает **токены**, разделённые пробелами;
- `«` выводит строку целиком, включая пробелы.

Пример

```
std::string a, b;
std::cin >> a >> b;   // reads two space-separated tokens

std::cout << "Hello, world!\n";
```

Возвращаемое значение:

- и », и « возвращают **ссылку на поток**;
- это позволяет строить цепочки: `std::cout « a « b « c;`.

4.6 Какие функции ввода и вывода можно использовать наравне с операторами? Как прочитать целиком строку, содержащую пробелы?

На практике вместе с операторами используют:

- `std::getline()` — чтение целой строки;
- функции посимвольного ввода, например `get()`, `peek()`, `ignore()`.

Чтобы прочитать строку с пробелами, используют:

```
std::string line;
std::getline(std::cin, line);
```

Примечание

`operator»` останавливается на пробеле, а `std::getline` читает до символа конца строки. Поэтому для текста с пробелами нужен именно `getline`.

4.7 Что такое состояние потока? Какие флаги его идентифицируют? Какие ошибки поправимы, а какие нет? Как сбросить ошибки?

Состояние потока

Состояние потока — это набор флагов, который показывает, можно ли продолжать чтение/запись и что пошло не так.

Основные флаги:

- `goodbit` — всё нормально;
- `eofbit` — достигнут конец файла/потока;
- `failbit` — операция не удалась, но поток может быть ещё пригоден;
- `badbit` — серьёзная ошибка потока.

По смыслу:

- **рабочее состояние**: `good()`;

- **поправимые ошибки**: обычно `failbit` и иногда `eofbit` как естественный конец чтения;
- **непоправимая ошибка**: `badbit`.

Пример

Поправимо «сломать» поток можно попыткой прочитать число из текста:

```
int x;
std::cin >> x;   // если введено "abc", будет failbit
```

Непоправимая ошибка связана с серьёзной проблемой устройства, файла или буфера.

Сброс флагов ошибок выполняет:

```
stream.clear();
```

После ошибок форматного ввода обычно дополнительно очищают «плохой» хвост строки:

```
std::cin.clear();
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
```

4.8 Что такое неявное приведение потока к bool? Как оно применяется в проверках вида «если поток»?

Поток в C++ можно неявно преобразовать к булевому значению. Это удобно для проверки, можно ли продолжать работу с потоком.

Поток как bool

Если поток преобразуется к `true`, то не установлены `failbit` и `badbit`; если к `false`, значит установлен хотя бы один из этих двух флагов.

Идиоматическая проверка:

```
if (s) { ... }
if (!s) { ... }
```

Важно: `if (s)` и `s.good()` — не одно и то же.

- `s.good()` истинно только при состоянии `goodbit` (вообще нет флагов ошибок);
- булево преобразование потока эквивалентно проверке «нет `failbit` и `badbit`»;
- поэтому связь с `good` такая: `s.good() => bool(s)`, но не наоборот.

4.9 Приведите идиоматические конструкции чтения до конца потока

Для строк:

```
std::string line;
while (std::getline(std::cin, line)) {
    // process line
}
```

Для токенов:

```
std::string word;
while (std::cin >> word) {
    // process word
}
```

Примечание

Это идиоматический стиль C++: сама операция чтения возвращает поток, и его булево значение показывает, удалось ли считать очередной элемент.

4.10 Как можно переназначить стандартные потоки ввода и вывода на другие устройства? Нужна ли перекомпиляция?

Стандартные потоки можно перенаправить на файлы или другие устройства средствами самой операционной системы или через перенаправление в shell.

Пример

```
program < input.txt > output.txt
```

Это означает:

- `stdin` берётся из файла `input.txt`;
- `stdout` идёт в файл `output.txt`.

Перекомпиляция программы для этого не нужна.

4.11 Что такое кодирование символов? Как решается проблема больших алфавитов? Как устроен Unicode?

Кодирование символов

Кодирование символов — это правило сопоставления символам числовых кодов, которые затем хранятся в памяти или передаются по каналу.

Проблема национальных алфавитов состоит в том, что обычной 8-битной таблицы недостаточно для всех символов.

Unicode решает задачу так:

- каждому символу соответствует кодовая позиция (*code point*);
- множество символов охватывает очень большое пространство кодов;
- символы группируются в плоскости (*planes*).

Примечание

Максимум для Unicode в современной модели — $0x10FFFF + 1 = 1\,114\,112$ кодовых позиций, то есть 2^{21} возможных code point-ов.

4.12 8-битные кодировки: примеры, ограничения, ASCII

Примеры 8-битных кодировок:

- ASCII как базовая часть;
- Windows-1251;
- KOI8-R;
- ISO-8859-1.

Ограничения:

- всего $2^8 = 256$ кодовых позиций;
- первые 128 позиций исторически совпадают с ASCII;
- этого недостаточно для всех языков мира.

Примечание

ASCII занимает первую половину многих 8-битных кодировок: 128 символов из 256.

Примечание

В стандарте Unicode символы ASCII занимают диапазон кодовых позиций `U+0000`–`U+007F` и входят в блок *Basic Latin*, который является частью *Basic Multilingual Plane (BMP)*.

4.13 Что такое `std::stringstream` и когда он нужен?

`std::stringstream` — это потоковая обёртка над строкой, которая одновременно является и входным, и выходным потоком.

- `std::istringstream` — только ввод из строки;
- `std::ostringstream` — только вывод в строку;
- `std::stringstream` — и ввод, и вывод в одну и ту же строку.

Пример

```
std::stringstream ss;
ss << 10 << ' ' << 20;

int a, b;
ss >> a >> b;
```

Такие потоки удобны для парсинга, форматирования и промежуточной обработки текста без реального файла или консоли.

4.14 Типы символов в C++ и символьные литералы

Основные типы символов:

- `char`;
- `signed char`;
- `unsigned char`;
- `wchar_t`;
- `char16_t`;
- `char32_t`.

Символьный литерал

Символьный литерал — это запись символа в коде, например `'a'`, `'\n'`.

Префиксы уточняют тип:

- `L'a'` — `wchar_t`;
- `u'a'` — `char16_t`;
- `U'a'` — `char32_t`;
- `u8'a'` — UTF-8-символ в кодовой форме UTF-8.

4.15 UTF-8 как кодировка переменной длины

UTF-8 — это кодировка переменной длины, в которой один символ Unicode кодируется от 1 до 4 байт.

Плюсы:

- совместимость с ASCII;
- экономичное хранение латиницы;
- широкая поддержка в современных системах.

Минусы:

- переменная длина усложняет случайный доступ;
- нужно аккуратно считать байты и символы;
- индексация по байтам не равна индексации по символам.

4.16 Статические C-массивы, `sizeof`, decay

C-массив

Статический C-массив — это последовательность элементов одного типа с фиксированным количеством элементов, заданным в момент объявления.

Пример

```
int a[5] = {1, 2, 3, 4, 5};
```

Число элементов можно получить как:

```
sizeof(a) / sizeof(a[0])
```

Примечание

Тип массива включает число элементов. `int[5]` и `int[7]` — это разные типы.

Decay

Decay — преобразование массива к указателю на первый элемент в большинстве контекстов.

Например, массив `int a[5]` часто преобразуется к `int*`. После decay теряется информация о размере массива.

4.17 C-строки и символ конца строки `'\0'`

C-string

C-строка — это массив символов `char`, заканчивающийся нулевым символом `'\0'`.

Пример

```
char s[] = "hello";
```

Здесь фактически хранится:

```
'h', 'e', 'l', 'l', 'o', '\0'
```

Разница между `char* arr` и `char* str` сама по себе не видна по типу: по указателю нельзя понять, ссылается ли он на обычный массив символов или на C-строку. Именно наличие `'\0'` делает массив строкой в стиле C.

Примечание

Символ `'\0'` — это не символ цифры ноль `'0'`, а специальный нулевой байт со значением 0.

4.18 Класс `std::string`: назначение, инициализация, методы, внутреннее представление

std::string

`std::string` — класс стандартной библиотеки для хранения и обработки динамических строк.

Инициализация:

```
std::string s1;
std::string s2 = "hello";
std::string s3("world");
std::string s4{"text"};
```

Часто используемые методы:

- `size()`, `length()`;
- `empty()`;
- `clear()`;
- `append()`;
- `substr()`;
- `c_str()`;
- `push_back()`;

Метод `c_str()` возвращает указатель на непрерывный буфер, заканчивающийся `'\0'`. Это нужно для взаимодействия с C-API.

Примечание

Стандарт гарантирует непрерывность памяти строки и наличие завершающего нулевого символа при вызове `c_str()`. Внутри объект может хранить данные в куче, а для коротких строк применять оптимизацию *short string optimization*.

4.19 Как читать токен и строку с пробелами в `std::string`?

Токен без пробелов:

```
std::string word;
std::cin >> word;
```

Строка с пробелами:

```
std::string line;
std::getline(std::cin, line);
```

Примечание

После чтения через » иногда перед `getline` нужно удалить символ перевода строки, например через `std::cin.ignore()`.

4.20 Связь между `sizeof(std::string)` и длиной строки `s.length()`

`sizeof(std::string)` — это размер самого объекта-обёртки, а не длина текста внутри него.

- `sizeof(std::string)` обычно постоянен для данной реализации;
- `s.length()` показывает число символов в строке;
- эти величины не обязаны быть связаны.

Примечание

Короткая строка может не выходить в кучу благодаря SSO, а длинная строка почти наверняка хранится отдельно. Но `sizeof` всё равно показывает размер объекта, а не выделенной под текст памяти.

4.21 Mutable или immutable? Capacity и length

`std::string` — это **mutable** тип: его содержимое можно менять.

- `length()` — текущая длина строки;
- `capacity()` — объём выделенной памяти, который может быть использован без дополнительного перераспределения.

Примечание

`capacity()` всегда не меньше `length()`, но может быть существенно больше.

4.22 Оптимизация коротких строк (SSO)

Short String Optimization

SSO — это оптимизация, при которой короткая строка хранится прямо внутри объекта `std::string`, без отдельного выделения памяти в куче.

Цель SSO:

- ускорить создание и копирование коротких строк;
- уменьшить число обращений к динамической памяти;
- сократить накладные расходы.

4.23 Преобразования строк и чисел в стиле C++

Подходы C++:

- `std::to_string(n)` — число в строку;
- `std::stoi`, `std::stol`, `std::stod` — строка в число;
- `std::stringstream` — универсальный и гибкий механизм;
- `std::from_chars`, `std::to_chars` — низкоуровневые и эффективные функции.

4.24 Конкатенация строк

Менее эффективные подходы:

- многократное сложение через `+` в цикле;
- создание большого числа временных объектов.

Более эффективные:

- `append()`;
- заранее резервировать память через `reserve()`;
- использовать `std::stringstream` для накопления результата.

Пример

```
std::string s;
s.reserve(1000);
s += "abc";
s.append("def");
```

4.25 `std::stringstream`: назначение, конкатенация, преобразование, тестирование

std::stringstream

`std::stringstream` — строковый поток, который одновременно является наследником и `std::istream`, и `std::ostream`.

Основные применения:

- построение строки через потоковую запись;
- преобразование числа в строку и обратно;
- тестирование функций, работающих с потоками, без файлов.

Пример

```
std::stringstream ss;
ss << "x = " << 42;

std::string text = ss.str();
```

Преобразование строки в число:

```
std::stringstream ss("123");
int x;
ss >> x;
```

Примечание

В модульных тестах удобно подменять файловый поток на `std::stringstream`: функция получает поток как аргумент и работает одинаково и с файлом, и со строкой в памяти.

4.26 Краткие ответы на контрольные вопросы

- **Что такое стандартный ввод и вывод?** Это `stdin` и `stdout`, в C++ обычно `std::cin` и `std::cout`.
- **Что такое чтение по словам?** Ввод через », который отделяет токены пробелами.

- **Что такое чтение строки с пробелами?** `std::getline()`.
- **Что такое `eof`?** Флаг конца потока, означающий достижение конца данных.
- **Как перебрать поток до конца?** Через `while (std::getline(...))` или `while (std::cin » x)`.
- **Можно ли перенаправить ввод/вывод без перекомпиляции?** Да, через перенаправление в командной строке.
- **Что такое Unicode?** Стандарт кодирования большого множества символов через code point-ы.
- **Что такое C-строка?** Массив `char`, заканчивающийся `'\0'`.
- **Что такое `std::string`?** Удобный динамический тип для строк с управлением памятью.
- **Что такое SSO?** Оптимизация хранения коротких строк внутри объекта.

5 Функциональная декомпозиция, ссылки и константность

5.1 Как решать большую задачу так, чтобы решение было корректным, проверяемым, воспроизводимым, переиспользуемым и читаемым?

Декомпозиция

Декомпозиция — это разбиение большой задачи на более маленькие, слабо связанные подзадачи, каждая из которых проще для понимания, реализации и проверки.

Для разработки программ это означает:

- разбивать решение на функции с ясной ответственностью;
- выделять интерфейс и реализацию;
- избегать дублирования логики;
- делать код тестируемым и воспроизводимым;
- использовать понятные имена и минимальные зависимости.

Хорошее решение обычно строится по схеме:

1. формулируем входы, выходы и инварианты;
2. выделяем подзадачи;
3. реализуем каждую подзадачу отдельно;
4. проверяем каждую часть независимо;
5. затем связываем части в целое.

Примечание

Именно функциональная декомпозиция делает программу не просто «написанной», а инженерно организованной: её можно читать, тестировать, менять и переиспользовать.

5.2 Что такое функция с точки зрения языка C++? Как определить функцию или процедуру? Какие основные компоненты функции?

Функция

Функция в C++ — это именованный блок инструкций, который может получать параметры, выполнять вычисления и, возможно, возвращать результат.

Процедура

Процедура — это функция, не возвращающая значение; в C++ это функция с типом возврата `void`.

Функция обычно состоит из следующих частей:

- **заголовок** — тип возврата, имя, список параметров;
- **тело** — блок инструкций;
- **инструкция** `return` — завершение функции и, если нужно, возврат значения.

Пример

```
int sum(int a, int b) {
    return a + b;
}

void printHello() {
    std::cout << "Hello\n";
}
```

Примечание

Функция — это базовый строительный блок программы: через функции реализуется функциональная декомпозиция и уменьшается связанность кода.

5.3 Что такое неполный тип `void`? Можно ли породить объект такого типа? Что такое `void*`?

void

`void` — это неполный тип, обозначающий отсутствие значения или отсутствие конкретного типа.

Объект типа `void` породить нельзя, потому что у него нет определённого размера и представления в памяти.

`void*` — это указатель на некий объект неопределённого типа. Такой указатель может хранить адрес, но сам по себе не позволяет разыменовывать его без приведения к конкретному типу.

- `void` используется как тип возврата процедур;
- `void*` используется как универсальный указатель низкого уровня;
- в C++ `void*` применяется реже, чем в C, потому что C++ предпочитает типобезопасные абстракции.

5.4 Свободные функции и функции-члены. Методы классов и статические функции-члены

Свободная функция

Free function или *non-member function* — функция, не принадлежащая классу.

Функция-член

Member function — функция, объявленная внутри класса и работающая с его объектом.

Пример

```
int add(int a, int b) { return a + b; }   // free function

struct Foo {
    void bar() const;                     // member function
    static void baz();                    // static member function
};
```

Методом называют функцию-член, обычно нестатическую. Статическая функция-член принадлежит классу как namespace-like сущность, а не конкретному объекту.

Примечание

Свободная функция хороша как композиционная схема: она получает данные, обрабатывает их и возвращает результат без скрытого состояния. Для свободных функций обычно нежелательны побочные эффекты.

5.5 Частые уточнения: `std::getline`, статическая функция-член и методы

`std::getline` — это **свободная функция** из стандартной библиотеки, а не метод `std::istream`.

Статическая функция-член (например, `Foo::baz()`) остаётся **функцией-членом**, но вызывается через имя класса:

```
Foo::baz();
```

Метод — это нестатическая функция-член. Пример метода у `std::string`:

```
std::string s = "abc";
auto n = s.size(); // метод size()
```

5.6 Что такое side-effect и почему он нежелателен для свободных функций? Как `static` вносит состояние?

Side effect

Side effect — побочный эффект, то есть изменение внешнего состояния: запись в поток, изменение глобальной переменной, модификация аргумента по ссылке, обращение к файлу и т.д.

Побочный эффект нежелателен для свободной функции, потому что он:

- ухудшает предсказуемость;
- усложняет тестирование;
- делает функцию менее переиспользуемой;
- затрудняет отладку.

`static` в теле функции позволяет сохранить состояние между вызовами:

```
int counter() {
    static int x = 0;
    return ++x;
}
```

Здесь функция уже не полностью «чиста»: она зависит от предыдущих вызовов.

5.7 Как устроено определение функции? Что такое prototype, caller и callee?

Определение функции

Определение функции — это описание её сигнатуры и тела.

Обычно в определении есть:

- имя функции;
- тип возврата;
- список формальных параметров;
- тело;
- при необходимости — `return`.

Prototype

Prototype — это объявление функции без тела, то есть её интерфейсная часть.

Caller и callee

Caller — вызывающая функция; *callee* — вызываемая функция.

Пример

```
int f(int);      // prototype
int g() {
    return f(10); // g is caller, f is callee
}
```

5.8 Список формальных параметров, заголовок и прототип

Список формальных параметров описывает вход функции. В прототипе имена параметров необязательны:

```
int sum(int, int);      // корректный прототип
int sum(int a, int b);  // тоже корректный прототип
```

Отличие:

- **заголовок функции** — часть определения, за которой следует тело;
- **прототип** — объявление без тела, интерфейсная часть для других модулей трансляции.

5.9 Функция: интерфейсная часть и реализация. Как это связано со сборкой программы?

Интерфейс функции — это то, что видит пользователь функции:

- имя;
- тип возврата;
- параметры;
- иногда спецификаторы, например `const` у member function.

Реализация — это тело функции.
Обычно распределение такое:

- интерфейс — в заголовочном файле или модуле интерфейса;
- реализация — в `.cpp` или модуле реализации.

Примечание

Компилятору часто достаточно интерфейса на этапе проверки вызова, но линкеру нужно полное определение функции, чтобы связать вызов с конкретным телом.

5.10 Точки выхода из функции: `return` в практике

Функция может иметь несколько точек выхода через `return`. Это допустимо, если улучшает читаемость (например, ранний выход для проверки ошибок).

Пример

```
int safeDiv(int a, int b) {
    if (b == 0) {
        return 0;
    }
    return a / b;
}
```

Для `void`-функции инструкция `return;` также допустима и завершает выполнение процедуры.

5.11 Признаки хорошей функции и принципы функциональной декомпозиции

Хорошая функция обычно:

- делает одну вещь;
- имеет ясное имя;
- имеет небольшой размер;
- имеет минимальное число параметров;
- по возможности не имеет побочных эффектов;
- легко тестируется;
- слабо связана с остальным кодом.

Принципы функциональной декомпозиции:

1. одна функция — одна ответственность;
2. сначала абстракция, потом детали;
3. минимизировать связи между функциями;
4. отделять вычисление от ввода-вывода;
5. избегать дублирования логики.

Примечание

Симптомы плохой функции: слишком длинная, делает несколько разных задач, имеет много параметров, содержит много ветвлений, трудно тестируется и сложно читается.

5.12 Реентерабельность, параметризуемость, связность и тестируемость

Реентерабельная функция

Функция реентерабельна, если её можно безопасно вызывать повторно и параллельно без скрытого общего изменяемого состояния.

Параметризуемость

Параметризуемость — степень, в которой поведение функции задаётся входными параметрами, а не глобальным контекстом.

Непараметризованный, но хороший пример: функция без входов, которая возвращает фиксированную доменную константу.

```
int maxScore() {
    return 100;
}
```

Высокая связность означает, что шаги внутри функции решают одну задачу. Низкая связанность между функциями означает минимум лишних зависимостей.

Тестируемость растёт, когда функция:

- имеет явные входы и выходы;
- не зависит от глобального состояния;
- не смешивает вычисления с вводом-выводом.

5.13 Принципы S.O.L.I.D.

S.O.L.I.D.

S.O.L.I.D. — набор принципов объектно-ориентированного и модульного проектирования.

- **S — Single Responsibility Principle**: у сущности должна быть одна причина для изменения.
- **O — Open/Closed Principle**: сущность должна быть открыта для расширения и закрыта для изменения.
- **L — Liskov Substitution Principle**: объекты наследника должны быть взаимозаменяемы с объектами базового типа.
- **I — Interface Segregation Principle**: лучше несколько узких интерфейсов, чем один широкий.
- **D — Dependency Inversion Principle**: зависимости должны строиться на абстракциях, а не на конкретных реализациях.

5.14 Формальные и фактические параметры. Сигнатура и перегрузка функций

Формальные и фактические параметры

Формальные параметры — переменные в объявлении функции. *Фактические параметры* — значения, передаваемые при вызове.

Пример

```
int sum(int a, int b);   // a, b - formal parameters
int x = sum(2, 3);       // 2, 3 - actual arguments
```

Сигнатура функции

Сигнатура — это характеристика функции, по которой различаются перегрузки: имя и типы параметров; для member functions часто также учитываются cv-квалификаторы.

Перегрузка

Overload — несколько функций с одним именем, но различающихся сигнатурой.

Примечание

Перегрузка не может отличаться только возвращаемым типом: для выбора нужной функции важны имя и типы параметров, а не результат. Также значения по умолчанию не создают отдельную перегрузку.

Примечание

Сигнатура нужна отдельно от полного заголовка, чтобы различать именно «форму вызова», а не детали реализации.

5.15 Модели взаимодействия функций

Взаимодействие функций удобно изображать:

- графом вызовов функций;
- UML sequence diagram;
- схемой потоков данных.

Примечание

Граф вызовов показывает структуру зависимости, а sequence diagram — порядок сообщений и переходов управления во времени.

5.16 CV-квалификаторы. `const` и `volatile`

CV-квалификаторы

CV-qualified тип — это тип с квалификаторами `const` и/или `volatile`.

`const` означает, что объект нельзя изменять через данное имя. `volatile` означает, что значение может измениться вне контроля программы, поэтому компилятор не должен агрессивно оптимизировать обращения.

Пример

```
const int a = 5;
int const b = 5;
```

`const int` и `int const` эквивалентны: различается только синтаксическая запись, а не смысл.

Примечание

`const` используется часто, `volatile` — редко, в основном для низкоуровневого взаимодействия с оборудованием или сигналами извне.

5.17 Константный объект: что сохраняется и что запрещается

Объект, объявленный через `const`, остаётся полноценным объектом языка: у него есть имя (если оно задано), адрес, тип и значение.

`const` влияет именно на операционную семантику: через данное имя нельзя выполнять модифицирующие операции над объектом.

Пример

```
const int limit = 100;
int x = limit;  // читать можно
// limit = 200; // нельзя: ошибка компиляции
```

Это не делает объект «нематериальным»: память у него есть, и адрес можно получить (при соблюдении правил типа и времени жизни).

5.18 Ссылка как ещё одно имя для объекта

Ссылка

Reference — это альтернативное имя для уже существующего объекта.

Объект может иметь:

- 0 имен, если доступен только через адрес;
- 1 имя, если существует обычная переменная;
- много имён, если есть ссылки, псевдонимы, параметры по ссылке и т.д.

Примечание

Мнемоника такая: «объекту нужно ещё одно имя, если старое имя недоступно, неудобно или его нужно передать в другой контекст без копирования».

5.19 Создание ссылки, binding, перепривязка, отложенное привязывание

Ссылка создаётся через `&` в объявлении:

```
int x = 10;
int& r = x;
```

Это называется *binding*: ссылка привязывается к объекту.

Примечание

Обычную ссылку перепривязать нельзя. После привязки она остаётся связанной с тем же объектом до конца жизни.

Отложенное привязывание обычной ссылки невозможно. Если нужно «позже связать» сущность с объектом, используют указатель, `std::optional<std::reference_wrapper<T>` или другой уровень абстракции.

5.20 Ссылки в range-based for

Ссылка в range-based for нужна, когда нужно модифицировать элементы диапазона без копирования:

```
for (auto& x : v) {
    x *= 2;
}
```

Примечание

Без `&` создавалась бы копия элемента, и изменения не повлияли бы на исходную коллекцию.

5.21 Ссылки как параметры функций

Передача по ссылке позволяет callee изменить объект caller:

```
void inc(int& x) {
    ++x;
}
```

Ссылка как двунаправленный портал

Неконстантная ссылка позволяет функции «видеть» и изменять объект caller напрямую.

Использование:

- для модификации аргумента;
- для возврата нескольких значений через параметры;
- для избежания копирования больших объектов.

Пример возврата нескольких результатов без агрегатов:

```
void minMax(int a, int b, int& mn, int& mx) {
    if (a < b) {
        mn = a;
        mx = b;
    } else {
        mn = b;
        mx = a;
    }
}
```

5.22 Передача по значению и по ссылке. Правило по умолчанию

Примечание

В C++ по умолчанию всё передаётся по значению: копируется.

Передача по значению:

- хороша для малых объектов;
- защищает исходный объект от изменения;
- создаёт копию, что может быть дорого.

Передача по ссылке:

- удобна для больших объектов;
- избегает копирования;
- позволяет изменять объект;
- требует аккуратности с временем жизни объекта.

5.23 Константная ссылка

Константная ссылка

Const reference — это ссылка только для чтения: через неё нельзя изменять объект.

Применение:

- передача больших объектов без копирования;
- защита от случайного изменения;
- чтение элементов в range-based for:

Пример

```
for (const auto& x : v) {
    std::cout << x << '\n';
}
```

Примечание

Константная ссылка особенно полезна для «больших» объектов. Для «малых» копирование часто дешевле, чем усложнение логики ссылок.

5.24 Что такое «большие» и «малые» объекты? Правило трёх указателей

Правило трёх указателей

Объект считается «большим», если его копирование потенциально дорого и он по размеру сравним с тремя указателями или более, либо если он владеет динамическим ресурсом.

Неформально:

- **малые объекты**: `int`, `double`, `char`, небольшие `enum`;
- **большие объекты**: `std::string`, контейнеры STL, крупные структуры, объекты с динамической памятью.

5.25 Идиоматические правила передачи объектов в функции

Обычно используют такие правила:

1. **большие объекты для чтения** — через **константную ссылку**;
2. **малые объекты для чтения — по значению**;
3. **объекты для чтения и записи** — через **неконстантную ссылку**.

Пример

```
void print(const std::string& s);  // read-only large object
int square(int x);                 // small object by value
void normalize(std::vector<double>& v); // read/write by reference
```

5.26 Краткие ответы на контрольные вопросы

- **Что такое функция?** Именованный блок инструкций с входом и, возможно, выходом.
- **Что такое `void`?** Неполный тип, обозначающий отсутствие значения.
- **Что такое свободная функция?** Не член класса.
- **Что такое статическая функция-член?** Функция, принадлежащая классу, но не конкретному объекту.
- **Что такое `const`?** Запрет на изменение через данное имя.
- **Что такое ссылка?** Ещё одно имя для объекта.
- **Можно ли перепривязать ссылку?** Нет.
- **Когда передавать по ссылке?** Когда нужен доступ к исходному объекту или надо избежать копирования.
- **Когда передавать по значению?** Когда объект маленький или копия дешева и нужна изоляция.
- **Когда нужна константная ссылка?** Когда объект большой, но его надо только читать.

6 Стандартная библиотека: основные концепции и линейные контейнеры

6.1 Что такое UB (неопределённое поведение) в C++ и какую опасность оно представляет? Приведите примеры.

UB

Undefined Behavior (UB) — это ситуация, при которой стандарт C++ не задаёт никакого поведения программы. После UB компилятор вправе делать что угодно: программа может работать «как будто нормально», может аварийно завершиться, а может начать вести себя непредсказуемо.

Опасность UB в том, что ошибка может проявляться не всегда и не в том месте, где она допущена. Из-за этого UB особенно трудно отлаживать.

Пример

Примеры UB:

```
int* p = nullptr;
*p = 10;                  // разыменование nullptr

int a[3] = {1, 2, 3};
std::cout << a[10];       // выход за границы массива

int x;
std::cout << x;           // использование неинициализированной переменной

int y = 1;
int z = y / 0;            // деление на ноль для целых типов
```

Примечание

UB — это не ошибка компилятора и не исключение. Это нарушение правил языка, после которого поведение программы уже не гарантируется.

6.2 Как и с помощью инструкций `typedef` и `using` определяются псевдонимы типов? Для чего это применяется?

Псевдоним типа

Псевдоним типа — это новое имя для уже существующего типа.

Классический синтаксис:

```
typedef unsigned long ulong;
```

Современный синтаксис:

```
using ulong = unsigned long;
```

- `typedef` — старый способ;

- `using` — более читаемый и удобный, особенно для шаблонов.

Применение:

- упрощение длинных типов;
- повышение читаемости;
- скрытие деталей реализации;
- облегчение сопровождения кода.

6.3 Какие основные компоненты входят в состав стандартной библиотеки?

Стандартная библиотека

Стандартная библиотека C++ — это набор готовых компонентов: контейнеров, алгоритмов, итераторов, строк, потоков, умных указателей, математических и служебных средств.

Основные блоки:

- контейнеры (`std::vector`, `std::array`, `std::list`, `std::map` и др.);
- алгоритмы (`std::sort`, `std::find`, `std::count`);
- итераторы;
- строки и строковые потоки;
- потоки ввода-вывода;
- служебные заголовки и утилиты.

6.4 Что такое контейнеры, алгоритмы, итераторы? Какова связь между ними?

Контейнер

Контейнер — это объект, который хранит набор элементов и предоставляет к ним доступ.

Итератор

Итератор — это абстракция, указывающая на элемент контейнера и позволяющая переходить между элементами.

Алгоритм

Алгоритм стандартной библиотеки — это обобщённая функция, которая работает с диапазонами элементов через итераторы.

Связь между ними:

- контейнер хранит данные;
- итератор даёт доступ к данным;
- алгоритм обрабатывает данные через итераторы.

Примечание

Именно итераторы делают возможным принцип: один алгоритм — множество реализаций контейнеров.

6.5 Какая парадигма программирования описывает концепцию: один алгоритм — множество реализаций?

Это **generic programming** — обобщённое, или шаблонное, программирование.

Примечание

В C++ эта парадигма реализована через шаблоны: один и тот же алгоритм может работать с разными типами данных и контейнерами, если они удовлетворяют нужному интерфейсу.

6.6 В каком стиле стандартные алгоритмы применяются к стандартным контейнерам: ООП или функциональном? Почему?

Стандартные алгоритмы применяются в **функциональном** стиле.

- алгоритм — это отдельная функция;
- контейнер передаётся как диапазон через итераторы;
- результат зависит от входных данных, а не от скрытого состояния объекта.

Пример

```
std::sort(v.begin(), v.end());
```

Здесь алгоритм `sort` не является методом `vector`. Он работает с диапазоном, заданным итераторами.

6.7 Чем схожи и чем различаются линейные и ассоциативные контейнеры?

Линейные контейнеры

Линейные контейнеры хранят элементы в последовательности.

Ассоциативные контейнеры

Ассоциативные контейнеры организуют доступ к элементам по ключу или по правилу упорядочения/хеширования.

Сходства:

- оба вида контейнеров хранят множество объектов;
- оба поддерживают стандартные операции доступа и обхода.

Различия:

- линейные контейнеры ориентированы на порядок следования элементов;
- ассоциативные — на поиск по ключу или по свойству упорядоченности;
- у линейных контейнеров обычно проще доступ по позиции;
- у ассоциативных — лучше поиск по смысловому признаку.

6.8 Чем схожи и чем различаются ассоциативные упорядоченные и неупорядоченные контейнеры?

Упорядоченные ассоциативные контейнеры поддерживают элементы в некотором порядке, обычно по сравнению ключей. **Неупорядоченные** используют хеш-таблицы.

Сходства:

- оба вида обеспечивают доступ по ключу;
- оба предназначены для быстрого поиска.

Различия:

- упорядоченные контейнеры сохраняют порядок по ключу;
- неупорядоченные работают через hash function;
- упорядоченные обычно обеспечивают логарифмический поиск;
- неупорядоченные — в среднем близки к константному.

6.9 `std::vector`: для чего применяется и в чём его назначение?

std::vector

`std::vector` — это динамически расширяемый массив, который хранит элементы подряд в памяти.

Назначение:

- хранить последовательность однотипных объектов;
- обеспечивать быстрый доступ по индексу;
- удобно наращиваться по мере добавления элементов.

Примечание

`vector` — базовый универсальный линейный контейнер STL.

6.10 std::vector: доступ к элементам через operator[] и at()

operator[] даёт доступ по индексу без проверки границ. at() делает доступ с проверкой границ и при ошибке выбрасывает исключение.

- v[i] — быстрее, но потенциально опасно;
- v.at(i) — безопаснее, но чуть медленнее.

Пример

```
std::vector<int> v{1, 2, 3};

int a = v[1];    // без проверки
int b = v.at(1); // с проверкой
```

6.11 std::vector: что возвращает operator[]?

operator[] возвращает **ссылку** на элемент:

- для неконстантного вектора — T&;
- для константного — const T&.

Примечание

Механизм именно ссылочный, потому что это доступ к уже существующему элементу внутри контейнера, а не копия элемента.

6.12 std::vector: варианты инициализации

Основные варианты:

```
std::vector<int> a;
std::vector<int> b{1, 2, 3};
std::vector<int> c(5);        // 5 элементов по умолчанию
std::vector<int> d(5, 42);    // 5 элементов со значением 42
```

Также можно инициализировать из другого диапазона:

```
std::vector<int> e(v.begin(), v.end());
```

6.13 std::vector: добавление и удаление элементов. push_back(), pop_back(), insert(), erase()

- push_back() — добавление в конец;
- pop_back() — удаление последнего элемента;
- insert() — вставка в произвольную позицию;
- erase() — удаление из произвольной позиции.

По эффективности:

- `push_back()` в среднем амортизированно быстро;
- `pop_back()` — быстро;
- `insert()` и `erase()` в середине обычно медленнее, потому что требуется сдвиг элементов.

Примечание

Самые дешёвые операции для `vector` — доступ по индексу и добавление/удаление с конца.

6.14 `std::vector`: распределение памяти, `reserve()`, `size()`, `capacity()`

- `size()` — число реально хранимых элементов;
- `capacity()` — объём выделенной памяти, доступный без новой аллокации;
- `reserve(n)` — заранее выделить память под минимум `n` элементов.

Пример

```
std::vector<int> v;
v.reserve(1000);
```

Это полезно, если заранее известно примерное число элементов.

Примечание

Зарезервировать память при конструировании можно не всегда напрямую через отдельный параметр конструктора, но можно сразу создать нужный размер или затем вызвать `reserve()`.

6.15 Что происходит при росте `vector`? Что такое переаллоцирование и как оно влияет на итераторы?

Когда `vector` не хватает `capacity()`, он обычно выделяет новый, больший буфер и переносит туда элементы. Это называется переаллоцированием.

- рост памяти обычно происходит не по одному элементу, а с запасом;
- стратегия роста зависит от реализации, но обычно она геометрическая, чтобы `push_back()` оставался амортизированно быстрым;
- при переаллоцировании все указатели, ссылки и итераторы на элементы вектора становятся невалидными.

Примечание

Именно поэтому `reserve()` полезен: он уменьшает число переаллоцирований и помогает сохранить валидность ссылок и итераторов на более длинном промежутке времени.

6.16 std::vector: тип элементов и шаблонный параметр

Тип элементов задаётся шаблонным параметром:

```
std::vector<int>
std::vector<std::string>
std::vector<MyType>
```

После создания объекта изменить тип элементов уже нельзя: тип вектора фиксируется на этапе компиляции.

Примечание

`std::vector<int>` и `std::vector<double>` — это разные типы.

6.17 std::vector: ограничения на тип элементов

Вектор может хранить типы, которые:

- можно размещать в памяти;
- можно копировать или перемещать при росте контейнера;
- можно разрушать;
- для некоторых конструкторов и операций — default constructable.

Default constructable

Тип называется *default constructable*, если его можно создать без аргументов.

Copyable

Тип называется *copyable*, если его можно копировать.

Примечание

Современный `vector` использует и перемещение, но в учебной модели удобно помнить: элемент должен быть совместим с копированием/перемещением и разрушением.

6.18 Можно ли создать std::vector<std::istream>?

Нет, обычный `std::vector<std::istream>` создать нельзя.

- `std::istream` — неперемещаемый и некопируемый тип потока;
- потоками обычно не владеют как значениями в контейнере;
- вместо этого используют указатели, ссылки, `std::reference_wrapper` или контейнеры умных указателей.

Пример

```
std::vector<std::istream*> streams;
```

6.19 Можно ли создать вектор векторов? Чем ограничена вложенность?

Да, можно:

```
std::vector<std::vector<int>> m;
std::vector<std::vector<std::vector<int>>> t;
```

Вложенность ограничена:

- памятью;
- удобством работы;
- требованиями к типам элементов на каждом уровне;
- глубиной, которую разумно поддерживать в программе.

6.20 Как `vector` хранит элементы: по значению, по ссылке или через указатель?

`std::vector` хранит элементы **по значению** внутри своего непрерывного буфера.

Примечание

Он не хранит «ссылки на исходные элементы». При вставке элементы могут быть перемещены при перераспределении памяти.

6.21 `std::vector` и `std::array`: сходства и различия

Сходства:

- оба контейнера хранят элементы подряд;
- оба обеспечивают быстрый доступ по индексу;
- оба поддерживают работу с итераторами.

Различия:

- `std::array` имеет фиксированный размер, известный на этапе компиляции;
- `std::vector` динамически растёт;
- `std::array` — обёртка над статическим массивом;
- `vector` управляет динамической памятью.

6.22 `std::array` и С-массивы: сходства и различия

Сходства:

- фиксированный размер;
- хранение элементов подряд в памяти;
- быстрый доступ по индексу.

Различия:

- `std::array` — полноценный объект со стандартным интерфейсом;
- С-массив при многих операциях деградирует до указателя;
- у `std::array` есть методы `size()`, `begin()`, `end()`;
- С-массив не имеет встроенного безопасного интерфейса.

Примечание

`std::array` обычно предпочтительнее С-массива в современном С++: он безопаснее и лучше интегрируется со стандартной библиотекой.

7 Стандартная библиотека: итераторы

7.1 С какой целью функция, получающая параметр по ссылке, может возвращать тот же параметр тоже по ссылке (pass-through)?

Pass-through

Pass-through — это приём, при котором функция получает объект по ссылке и возвращает ссылку на тот же объект, не создавая копию.

Такой подход нужен, чтобы:

- не копировать объект;
- сохранить возможность цепочек вызовов;
- передать объект «сквозь» функцию без потери семантики ссылки.

Пример

```
int& pass(int& x) {
    return x;
}
```

Примечание

Pass-through полезен, когда функция выступает как промежуточное звено в цепочке преобразований, но не хочет забирать владение объектом и не должна его копировать.

7.2 Как условно объекты подразделяют на «большие» и «небольшие»? Какой критерий для этого применяется?

Большие и небольшие объекты

Объекты условно называют **небольшими**, если их копирование дешево, и **большими**, если копирование потенциально дорого.

Практический критерий — **правило трёх указателей**: объект считается «большим», если он по смыслу содержит более чем несколько машинных слов, владеет динамической памятью или его копирование требует заметной работы.

- **Небольшие**: `int`, `char`, `double`, маленькие `enum`.
- **Большие**: `std::string`, `std::vector`, крупные структуры, объекты с ресурсами.

7.3 Какие варианты последовательного перебора элементов линейного контейнера существуют?

Для линейного контейнера, например `std::vector`, обычно используют три способа:

- по индексу через `operator[]`;
- циклом `range-based for`;
- через итераторы.

Пример

```
std::vector<int> v{1, 2, 3};

for (std::size_t i = 0; i < v.size(); ++i) {
    std::cout << v[i] << ' ';
}

for (auto x : v) {
    std::cout << x << ' ';
}

for (auto it = v.begin(); it != v.end(); ++it) {
    std::cout << *it << ' ';
}
```

7.4 Для чего необходим механизм косвенной адресации, основанный на итераторах?

Итератор

Итератор — это объект косвенной адресации, который указывает на элемент коллекции и позволяет работать с ним без знания внутреннего устройства контейнера.

Итераторы нужны для:

- унификации доступа к разным контейнерам;
- отделения алгоритма от конкретного контейнера;
- обхода элементов в стандартных алгоритмах;
- абстракции «позиции» в коллекции.

Примечание

Итератор связывает контейнер и алгоритм: контейнер хранит элементы, итератор указывает на них, алгоритм их обрабатывает.

7.5 Сколько итераторов необходимо для представления одного элемента, двух элементов, n элементов?

Один итератор указывает на один элемент. Но для представления **диапазона** обычно используют **два** итератора: начало и конец.

- один элемент можно представить как полуинтервал `[it, it + 1)`;
- два элемента — `[it, it + 2)`;
- `n` элементов — `[first, last)`, где расстояние между ними равно `n`.

7.6 Как с помощью итераторов представляется range (диапазон) объектов?

Range

Range — это совокупность элементов, представляемая парой итераторов в виде полуинтервала `[first, last)`.

Здесь:

- `first` указывает на первый элемент диапазона;
- `last` указывает **на элемент сразу после последнего**.

Примечание

Такое представление удобно, потому что пустой диапазон задаётся равенством `first == last`, а длина диапазона выражается как расстояние между итераторами.

7.7 Что общего и что различается у итераторов `it1` и `it2` в диапазоне `[it1, it2)`?

Общее:

- оба относятся к одному диапазону;
- оба должны быть совместимы по типу и контейнеру;
- между ними определён порядок следования.

Различия:

- `it1` — итератор на первый элемент диапазона;
- `it2` — *past-the-end iterator*, то есть указывает *после* последнего элемента;
- `it1` обычно разыменовывается, `it2` — нет.

7.8 Допустимо ли разыменование `it1` и `it2` в диапазоне `[it1, it2)`?

- `it1` можно разыменовать, если диапазон не пустой и итератор валиден;
- `it2` разыменовывать нельзя, потому что это итератор «за последним».

Примечание

`it2` — это не элемент коллекции, а специальная граница диапазона. Именно поэтому его называют *sentinel* или *past-the-end iterator*.

7.9 Какие операции применимы к подавляющему большинству итераторов? Как умеют «двигаться» итераторы?

Базовые операции:

- конструирование;
- копирование;
- присваивание;
- разыменование `*it`;
- сравнение `it == it2`, `it != it2`;
- движение вперёд через `++it`.

Итераторы представляют «позицию» в контейнере и обычно перемещаются на соседний элемент.

7.10 Чем различаются свободная функция `std::begin(c)` и метод `c.begin()`?

`c.begin()` — это метод самого контейнера. `std::begin(c)` — свободная функция, которая вызывает подходящий `begin` для объекта `c`.

- метод удобен для контейнеров с собственным интерфейсом;
- свободная функция работает более универсально, в том числе с массивами;
- `std::begin` лучше вписывается в обобщённый код.

Пример

```
auto it1 = v.begin();
auto it2 = std::begin(v);
```

7.11 Какие типы итераторов встроены в стандартные контейнеры? Для чего несколько разновидностей?

Обычно контейнеры дают:

- `iterator`;
- `const_iterator`;
- `reverse_iterator`;
- `const_reverse_iterator`.
- `iterator` — для чтения и изменения;
- `const_iterator` — только для чтения;
- `reverse_iterator` — обход в обратном направлении;
- `const_reverse_iterator` — обратный обход без изменения элементов.

Примечание

Несколько разновидностей нужны для различной семантики доступа: чтение, запись, прямой обход, обратный обход.

7.12 Какие категории итераторов выделяли до стандарта C++20?

До C++20 обычно выделяли:

- **input iterators**;
- **output iterators**;
- **forward iterators**;
- **bidirectional iterators**;
- **random access iterators**.

- input — чтение последовательности вперёд;
- output — запись последовательности вперёд;
- forward — движение только вперёд;
- bidirectional — вперёд и назад;
- random access — произвольный доступ, прыжки на несколько позиций и арифметика итераторов.

Примечание

Категории чтения/записи и категории перемещения частично ортогональны, а по «силе перемещения» действует вложенность: random access сильнее bidirectional, а bidirectional сильнее forward.

7.13 Какие типы итераторов использует `std::vector`? А `std::list`?

- `std::vector` — random access iterators;
- `std::list` — bidirectional iterators;
- многие ассоциативные контейнеры — bidirectional;
- некоторые односвязные структуры — forward.

Примечание

Чем богаче операции контейнера, тем сильнее категория его итераторов.

7.14 Как итерировать от первого к «запоследнему» элементу и обратно? Какие операции при этом используются?

Прямой обход:

```
for (auto it = c.begin(); it != c.end(); ++it) {
    ...
}
```

Обратный обход:

```
for (auto it = c.rbegin(); it != c.rend(); ++it) {
    ...
}
```

Если есть bidirectional iterator, можно двигаться назад через `-it`. Если есть random access iterator, доступны также `it += n` и `it + n`.

Четыре типичных сценария из контрольных вопросов:

- от первого к запоследнему: `it = begin(); ++it ... it == end()`;
- после достижения `end()` вернуться к первому: просто присвоить `it = begin()`;
- от последнего к передпервому: `it = rbegin(); ++it ... it == rend()` или через `-` для bidirectional;
- после достижения начала в обратном обходе вернуться к последнему: `it = rbegin()` (или `it = -end()` для bidirectional).

7.15 Движение итераторов: `++it`, `it++`, `it += n`, `it + n`

- `++it` — перейти к следующему элементу;
- `it++` — постинкремент, вернуть старое значение и затем сдвинуться;
- `it += n` — сдвиг на `n` позиций вперёд;
- `it + n` — получить новый итератор, смещённый на `n`.

Примечание

`++it` обычно предпочтительнее, чем `it++`, потому что постинкремент может создавать лишнюю копию.

7.16 Как сравнивать итераторы при переборе коллекции: `it != c.end()` или `it < c.end()`?

Предпочтительно использовать:

```
it != c.end()
```

Потому что:

- это поддерживается большинством категорий итераторов;
- это естественный способ проверки конца диапазона;
- `<` доступно не для всех итераторов.

Примечание

Сравнение через `<` допустимо только для random access iterator, например у `std::vector`. Но универсальный код должен опираться на `!=`.

7.17 Можно ли копировать итератор `c.begin()`? Можно ли его разыменовать? Можно ли двигать вперёд?

Да, копировать можно. Да, если он указывает на валидный элемент, его можно разыменовать. Да, можно двигать вперёд, если контейнер и категория итератора это позволяют.

Примечание

Итератор — это полноценный объект, который удобно передавать по значению.

7.18 Можно ли копировать итератор `c.end()`? Можно ли его разыменовать? Можно ли двигать назад?

Копировать `c.end()` можно. Разыменовывать нельзя. Двигать назад можно только если категория итератора это допускает, например для bidirectional или random access.

Примечание

`end()` — это допустимая граница диапазона, но не элемент диапазона.

7.19 Какие операции допустимы к итератору на валидный элемент и к итератору *past-the-end*?

Для итератора на валидный элемент обычно допустимы:

- разыменование;
- инкремент;
- сравнение;
- в зависимости от категории — декремент и прыжки.

Для *past-the-end* итератора:

- разыменование запрещено;
- сравнение допустимо;
- движение назад возможно только для соответствующих категорий и только если это не нарушает валидность.

7.20 Какой тип итератора следует выбирать при разработке нового алгоритма: минимально достаточный или максимально сильный?

Следует требовать **минимально достаточную** категорию итератора, которая реально нужна алгоритму.

- если алгоритму достаточно однопроходного обхода, не надо требовать random access;
- если нужен только просмотр вперёд, не надо требовать bidirectional;
- чем слабее требование, тем больше контейнеров сможет использовать алгоритм.

Примечание

Это важный принцип обобщённого программирования: алгоритм должен быть как можно менее требовательным.

7.21 Что происходит с итератором после удаления элемента из коллекции по позиции этим итератором?

Итератор, по которому удалили элемент, обычно становится невалидным.

- после `erase(it)` старый `it` использовать нельзя;
- обычно `erase` возвращает новый валидный итератор на следующий элемент;
- его нужно сохранить и использовать дальше.

Пример

```
auto it = v.erase(it); // старый it уничтожен, используется возвращённый
```

7.22 Когда итераторы остаются валидными, а когда нет?

Правило зависит от контейнера и операции.

- Для `std::vector` вставка или удаление в середине сдвигает элементы, а при переаллоцировании инвалидируются все итераторы, ссылки и указатели на элементы.
- Для `std::list` и похожих связных структур удаление инвалидирует обычно только итератор на удалённый элемент, остальные остаются валидными.
- Для `std::set` и `std::map` вставка обычно не инвалидирует существующие итераторы, а удаление инвалидирует только итератор на удалённый элемент.

Примечание

Если алгоритм хранит итератор на элемент контейнера, нужно знать правила инвалидирования именно для этого контейнера. Иначе можно продолжить работу с уже невалидной позицией.

7.23 Краткие ответы на контрольные вопросы

- **Зачем pass-through?** Чтобы вернуть ссылку на тот же объект без копирования.
- **Что такое iterator?** Объект косвенной адресации и связующий элемент между контейнером и алгоритмом.
- **Сколько итераторов нужно для диапазона?** Обычно два: начало и конец.

- **Почему `[first, last)`?** Потому что конец не включается, это упрощает пустые диапазоны и алгоритмы.
- **Что такое `end()`?** Итератор на позицию после последнего элемента.
- **Почему `it != end()` лучше, чем `it < end()`?** Потому что универсальнее.
- **Какие категории итераторов существуют?** Forward, bidirectional, random access.
- **Что делать с итератором после `erase`?** Считать старый итератор недействительным и использовать возвращённый.

8 Косвенная адресация

8.1 Что такое косвенная адресация и для чего она применяется?

Косвенная адресация

Indirect addressing — это способ доступа к объекту не по его прямому имени, а через некоторую промежуточную сущность: указатель, ссылку, итератор, `string_view` и т.п.

Косвенная адресация нужна, когда:

- объект должен быть доступен не только по своему имени;
- нужно передать объект в функцию без копирования;
- нужно работать с «чужим» объектом через абстракцию;
- нужно обойти коллекцию или часть коллекции;
- требуется отделить интерфейс от конкретного хранения данных.

8.2 Какие механизмы языка C++ и компоненты стандартной библиотеки вовлекаются при косвенной адресации?

Основные механизмы:

- указатели;
- ссылки;
- итераторы;
- `std::string_view`;
- в более широком смысле — умные указатели и обёртки над ссылками.

Примечание

Все эти средства выполняют одну идею: дать доступ к объекту через другое имя или через позицию, не копируя сам объект без необходимости.

8.3 Какое количество объектов как правило задействовано при косвенной адресации: при использовании указателя? ссылки? итератора?

Обычно:

- **указатель** связывает два объекта: сам указатель и объект, на который он указывает;
- **ссылка** также связывает два объекта: имя-ссылку и объект-цель;
- **итератор** обычно тоже связан с двумя сущностями: самим итератором и элементом коллекции, на который он указывает.

8.4 Косвенная адресация и константность. Один объект — две разновидности константности. Два объекта — четыре разновидности константности

Когда есть один объект и одна косвенная сущность, можно рассматривать две независимые характеристики:

- можно ли изменять **объект** через это имя;
- можно ли изменять саму **косвенную сущность** как объект языка.

Если рассматривать пару «объект + косвенность», то возникают четыре комбинации:

1. объект изменяем, косвенность изменяема;
2. объект неизменяем, косвенность изменяема;
3. объект изменяем, косвенность неизменяема;
4. объект неизменяем, косвенность неизменяема.

Примечание

Для указателей и итераторов эти варианты хорошо видны. Для ссылок картина беднее, потому что ссылка в C++ не перепривязывается.

8.5 Указатели: «левоконстантный», «правоконстантный» и «дваждыконстантный» указатель

Пусть есть указатель `T* p`. Тогда константность может относиться:

- к тому, можно ли менять **содержимое**, на которое указывает `p`;
- к тому, можно ли менять **сам указатель** `p`.

Левоконстантный указатель

`const T* p` или `T const* p` — указатель на константный объект. Нельзя менять объект через `p`, но можно переназначать сам указатель.

Правоконстантный указатель

`T* const p` — сам указатель константен. Можно менять объект через `p`, но нельзя переназначить `p` на другой объект.

Дваждыконстантный указатель

`const T* const p` — и сам указатель, и объект под ним константны. Нельзя ни менять объект через `p`, ни переназначать `p`.

8.6 Ссылки: константность и неконстантность

Неконстантная ссылка

`T& r` — ссылка, через которую можно менять объект.

Константная ссылка

`const T& r` — ссылка только для чтения: через неё нельзя изменять объект.

Для ссылок важное отличие от указателей такое:

- ссылка не может быть пустой;
- обычную ссылку нельзя перепривязать;
- поэтому у ссылки нет полноценной аналогии с «правоконстантным» указателем.

Примечание

В разговорной модели можно говорить, что у ссылки есть константность относительно объекта, на который она смотрит, но сама ссылка как «переменная-сущность» не ведёт себя как указатель.

8.7 Итераторы: `const_iterator`, `const iterator` и `const const_iterator`

У контейнеров STL обычно есть:

- `iterator` — итератор, через который можно менять элемент;
- `const_iterator` — итератор, через который можно только читать элемент;
- `const iterator` — константный объект типа `iterator`;
- `const const_iterator` — константный объект типа `const_iterator`.

Примечание

`const iterator` и `const_iterator` — это не одно и то же:

- `iterator` / `const iterator` — речь о константности самого итераторного объекта;
- `const_iterator` — речь о типе, который не позволяет менять элемент коллекции.

8.8 Почему для «левоконстантного» итератора требуется отдельный тип `const_iterator`?

Для указателей константность можно выразить прямо квалификатором `const`. Для итераторов этого недостаточно, потому что итератор — это уже не просто указатель, а абстрактный объект со своим интерфейсом.

Поэтому в стандартной библиотеке вводят отдельный тип:

- `iterator` — для изменения элемента;
- `const_iterator` — для только чтения элемента.

Примечание

Это нужно, чтобы сам интерфейс итератора явно запрещал изменение элемента через «левоконстантный» доступ. Иначе был бы риск получить внешний объект, который выглядит как константный, но всё ещё позволяет менять данные через разыменование.

8.9 Краткие ответы на контрольные вопросы

- **Что такое косвенная адресация?** Доступ к объекту через промежуточную сущность.
- **Зачем она нужна?** Для передачи, обхода и абстрагирования доступа.
- **Какие механизмы используются?** Указатели, ссылки, итераторы, `string_view`.
- **Сколько объектов обычно участвует?** Обычно два: косвенная сущность и объект-цель.
- **Что такое `const T*`?** Указатель на константный объект.
- **Что такое `T* const`?** Константный указатель.
- **Что такое `const T&`?** Константная ссылка.
- **Что такое `const_iterator`?** Итератор только для чтения элементов.
- **Зачем отдельный `const_iterator`?** Чтобы запретить изменение элемента через итератор на уровне типа.

9 Класс std::string_view

9.1 Ключевое слово auto: назначение, правила применения, ограничения

auto

`auto` — это ключевое слово, которое позволяет компилятору вывести тип переменной из инициализатора.

Основные назначения:

- сокращение записи длинных и сложных типов;
- уменьшение жёсткой привязки к конкретной реализации типа;
- удобство при работе с итераторами, шаблонами и обобщённым кодом.

Пример

```
auto x = 42;              // int
auto y = 3.14;            // double
auto it = v.begin();      // тип итератора выводится автоматически
```

Ограничения:

- `auto` должен иметь инициализатор;
- он не задаёт тип сам по себе, а только выводит его;
- при использовании `auto` важно понимать, что вывод может убрать ссылки и `const`, если не указать `auto&` или `const auto&`.

Примечание

На практике `auto` особенно полезен там, где тип либо слишком длинный, либо несущественен для понимания кода.

9.2 Строковые литералы, глобальная доступность и слияние одинаковых литералов

Строковый литерал

Строковый литерал — это текст в двойных кавычках, например `"hello"`.

Строковые литералы:

- имеют статическое время жизни;
- обычно доступны на всём протяжении выполнения программы;
- являются глобально доступными в смысле времени жизни и адресуемости.

Примечание

Одинаковые строковые литералы компилятор или линкер может *сливать* (*merge*): несколько одинаковых литералов могут указывать на один и тот же участок памяти. Но полагаться на это как на строгое правило языка не следует.

9.3 `std::string_view` как разновидность косвенной адресации

std::string_view

`std::string_view` — это невладеющее представление последовательности символов: он хранит ссылку на данные и длину, но не управляет памятью.

С точки зрения косвенной адресации это:

- не копия строки;
- не владелец данных;
- лёгкая обёртка над «окном» в строку;
- средство доступа к подстроке без выделения новой памяти.

9.4 Какие объекты нужны для представления Null-terminated строки?

Для представления Null-terminated строки обычно достаточно:

- одного массива символов `char[]`;
- или одного указателя `char*`, если сам массив уже существует и конец строки обозначен символом `'\0'`.

Пример

```
char s[] = "hello";
const char* p = s;
```

Примечание

Смысл Null-terminated строки задаётся именно наличием завершающего нулевого символа, а не отдельным объектом длины.

9.5 Какие объекты нужны для представления суффикса, префикса и инфикса Null-terminated строки?

Для подстрок возможны разные представления.

- **Префикс**: достаточно указателя на начало строки, если подстрока заканчивается `\0`, либо указателя + длина, если конец не совпадает с физическим концом.
- **Суффикс**: часто достаточно указателя на начало суффикса, если строка уже null-terminated.
- **Инфикс**: обычно нужны либо два указателя `[first, last)`, либо указатель + длина.

Примечание

Для общего случая подстроки удобнее хранить не только начало, но и длину. Тогда можно обращаться и к суффиксу, и к инфиксу без поиска конца.

9.6 Два указателя против указателя + длина

Есть два основных способа представить подстроку:

- **пара указателей** `[first, last)`;
- **указатель + длина** `(ptr, size)`.

Пара указателей хорошо соотносится с концепцией range в C++:

$$[first, last)$$

где `last` указывает на элемент **после** последнего.

Плюсы:

- естественная модель диапазона;
- удобна для алгоритмов;
- не нужно отдельно хранить длину.

Минусы:

- длину приходится вычислять как расстояние между указателями;
- не всякий код удобно писать в таком виде.

Указатель + длина:

- удобно хранить длину явно;
- быстро проверять размер;
- удобно для строковых view.

Минусы:

- нужно следить за согласованностью начала и длины;
- это не совсем та же форма, что классический range `[first, last)`.

9.7 Передача и возврат `std::string_view` из функций

`std::string_view` обычно передают и возвращают **по значению**.

Пример

```
void print(std::string_view sv) {
    std::cout << sv << '\n';
}
```

```
std::string_view head(std::string_view sv) {
    return sv.substr(0, 3);
}
```

Примечание

Такой объект считается «нетяжёлым»: он обычно хранит только указатель и длину, поэтому копировать его дёшево.

9.8 Конструирование и копирование `std::string_view`. Почему это невладеющий тип?

`std::string_view` можно создать из:

- строкового литерала;
- С-строки;
- `std::string`;
- другого `std::string_view`.

Пример

```
std::string s = "hello";
std::string_view v1 = s;
std::string_view v2 = "world";
std::string_view v3 = v1;
```

Он **не владеет** данными:

- не выделяет память под строку;
- не освобождает память;
- лишь ссылается на уже существующий буфер.

Примечание

Из-за этого главное ограничение `string_view`: он не должен жить дольше, чем данные, на которые ссылается.

9.9 Ограничения `std::string_view` как невладеющего типа

Поскольку `string_view` не владеет данными, возникают ограничения:

- нельзя сохранять view дольше, чем живёт исходная строка;
- нельзя использовать его как замену владению;
- нужно быть осторожным с временными объектами;
- модификация исходной строки может сделать view невалидным.

Пример

```
std::string_view bad() {
    std::string s = "temp";
    return s; // dangling view
}
```

9.10 Ограничение std::string_view при работе со стандартной библиотекой

std::string_view не всегда можно передавать туда, где ждут const char*.

- если функция принимает именно C-строку, ей нужен нуль-терминированный буфер;
- string_view гарантирует диапазон символов, но не всегда гарантирует наличие завершающего '\0' в конце именно подстроки;
- поэтому иногда требуется явное преобразование в std::string или использование .data() с осторожностью.

Примечание

Если API библиотеки требует const char* и предполагает нуль-терминацию, string_view можно использовать не во всех случаях без дополнительной проверки.

9.11 Ключевые операции над строками с помощью std::string_view

Основные операции:

- size(), length();
- empty();
- substr();
- remove_prefix();
- remove_suffix();
- compare();
- starts_with(), ends_with() в современных стандартах.

Пример

```
std::string_view sv = "hello world";
auto prefix = sv.substr(0, 5);
sv.remove_prefix(6);
```

9.12 Что происходит, если модифицировать подстроку через `std::string_vi`

`std::string_view` сам по себе не модифицирует данные, потому что он не владеет памятью и не является изменяемой строкой.

- можно изменить сам **view**: например, сдвинуть начало через `remove_prefix()`;
- нельзя изменить символы исходной строки через `string_view`;
- если нужна модификация текста, нужно работать со `std::string` или другим изменяемым буфером.

Примечание

После изменения самого view это уже тот же объект `string_view`, но он начинает указывать на другой участок того же исходного буфера.

9.13 Краткие ответы на контрольные вопросы

- **Какие объекты нужны для null-terminated строки?** Обычно один массив `char[]` или указатель на него.
- **Какие объекты нужны для подстроки?** Для общего случая удобны либо пара указателей, либо указатель + длина.
- **Как связаны пары `char*` и `char* + size_t`?** Это два способа задать границы подстроки; они не полностью взаимозаменяемы без потери информации о конце или длине.
- **Как передают `string_view`?** Обычно по значению.
- **Глобально ли доступен строковый литерал?** Да, по времени жизни он статический.
- **Гарантирует ли `string_view` нуль-терминацию?** Нет, только если view охватывает строку целиком и источник это обеспечивает.
- **Можно ли всегда подставить `string_view` вместо `const char*`?** Нет, только когда требуется нестрогая работа с диапазоном или когда гарантирована нуль-терминация.
- **Можно ли модифицировать подстроку через `string_view`?** Сам view — да (например, сдвиг границ), сами символы — нет.

10 Ассоциативные контейнеры `std::set` и `std::map`

10.1 Какие основные характеристики имеют ассоциативные контейнеры?

Ассоциативные контейнеры – это контейнеры, в которых элементы хранятся не в порядке вставки, а в соответствии с некоторым правилом упорядочивания. Для стандартных контейнеров `std::set` и `std::map` таким правилом обычно является сравнение по `operator<` (или по пользовательскому компаратору).

Ассоциативный контейнер

Ассоциативный контейнер хранит элементы в упорядоченном виде и поддерживает эффективный поиск по ключу. Основные операции – поиск, вставка и удаление – обычно имеют логарифмическую сложность.

Ключевые свойства:

- элементы упорядочены по строгому слабому порядку;
- поиск выполняется существенно быстрее, чем линейный просмотр;
- контейнер не является последовательным: порядок обхода задаётся сортировкой, а не временем вставки;
- обычно не допускаются дубликаты ключей;
- итерация по контейнеру идёт в отсортированном порядке.

10.2 Что такое `std::set`? Каковы основные операции и ограничения?

`std::set` – это множество уникальных значений. Один и тот же элемент не может присутствовать в контейнере дважды.

`std::set`

`std::set<T>` – ассоциативный контейнер, хранящий элементы типа `T` в упорядоченном виде без дубликатов.

Основные операции:

- `find(x)` – поиск элемента;
- `insert(x)` – вставка элемента;
- `erase(x)` или `erase(it)` – удаление элемента;
- `begin()`, `end()` – обход контейнера.

Если вставляется значение, уже присутствующее в `set`, то новый элемент не добавляется: контейнер остаётся без изменений.

Примечание

В учебной формулировке часто говорят, что элементы `set` должны быть *default constructable*, *copyable* и *comparable*. В более точном стандарте важно, чтобы элементы можно было сравнивать с помощью компаратора, задающего *strict weak ordering*. На практике это обычно означает наличие корректного `operator<` или пользовательского сравнения.

10.3 Какие требования накладываются на тип элементов `std::set` и почему?

Ключевое требование: элементы должны быть сравнимы через компаратор контейнера так, чтобы задавался строгий слабый порядок.

Это необходимо, потому что `set` хранит элементы в упорядоченной древовидной структуре и использует сравнение для:

- поиска пути в дереве;
- проверки эквивалентности ключей;
- поддержания корректного порядка при вставке и удалении.

На практике элемент обычно должен быть копируемым или перемещаемым и корректно разрушаемым, чтобы контейнер мог безопасно управлять временем жизни узлов.

10.4 Как итерироваться по `std::set`? К какой категории относятся итераторы?

Итерирование по `std::set` происходит в отсортированном порядке, то есть в том порядке, в каком элементы лежат в дереве поиска.

Пример:

```
std::set<int> s = {3, 1, 4};
for (auto it = s.begin(); it != s.end(); ++it) {
    std::cout << *it << ' ';
}
```

Итераторы `std::set` относятся к категории **bidirectional iterator**. Это означает, что:

- можно двигаться вперёд через `++it`;
- можно двигаться назад через `-it`;
- нельзя делать произвольный доступ, как у random access iterator, то есть нельзя писать `it + n`.

Причина в том, что `set` обычно реализован как дерево, а не как массив.

10.5 Как искать элемент в `std::set`? Что возвращает `find()`?

Основная операция поиска – `find(x)`.
Она возвращает итератор:

- на найденный элемент, если он есть в контейнере;
- на `end()`, если элемента нет.

Это значение можно использовать:

- для проверки наличия элемента;
- для удаления через `erase(it)`;
- для доступа к найденному элементу через разыменование итератора.

Идиома проверки наличия:

```
if (s.find(x) != s.end()) {
    // x есть в множестве
}
```

10.6 Как работают `insert()` и `erase()` в `std::set`? Почему их сложность связана с `find()`?

Обе операции опираются на поиск места в дереве.

- `insert(x)` сначала находит позицию, куда должен попасть элемент, а затем вставляет его.
- `erase(x)` сначала находит нужный элемент, а затем удаляет его и восстанавливает свойства дерева.

Поэтому алгоритмическая сложность этих операций определяется сложностью поиска:

$$O(\log n)$$

при типичной реализации на сбалансированном дереве.

Если удаление или вставка были бы возможны без поиска, это всё равно не отменяло бы необходимость определить место элемента в упорядоченной структуре. Именно поэтому `find()` является базовой операцией, от которой зависят остальные.

10.7 Как обычно реализован `std::set`?

Обычная реализация – это **сбалансированное бинарное дерево поиска**, чаще всего красно-чёрное дерево (*red-black tree*).

Следствия такой реализации:

- поиск, вставка и удаление имеют сложность `O(log n)`;
- обход контейнера идёт в отсортированном порядке;
- память расходуется на узлы дерева, а не на плотный массив.

Примечание

Именно дерево, а не массив, объясняет тип итераторов: можно идти к следующему и предыдущему элементу, но нельзя обращаться к элементу по индексу.

10.8 Что такое `std::pair`? Почему он нужен?

`std::pair` – это вспомогательный класс для хранения двух значений, обычно разных типов.

`std::pair`

`std::pair<T1, T2>` – агрегат, представляющий упорядоченную двойку значений `first` и `second`.

Его используют, когда нужно хранить связанные данные:

- ключ и значение;
- координаты;
- две связанные величины.

Свойства `pair`:

- это кортеж из двух элементов;
- удобно создаётся через фигурные скобки или `std::make_pair`;
- сравнение идёт лексикографически: сначала `first`, затем `second`.

Пример

```
std::pair<int, std::string> p{1, "one"};
std::cout << p.first << ' ' << p.second;
```

10.9 Что такое `std::map`? Чем он отличается от `std::set<std::pair<K,V»`?

`std::map` – это ассоциативный контейнер, который хранит пары `(key, value)`. Ключи уникальны.

`std::map`

`std::map<K, V>` – контейнер, в котором каждому ключу типа `K` соответствует значение типа `V`. Ключи упорядочены, а для каждого ключа хранится не более одного значения.

Различие между `std::map<K, V>` и `std::set<std::pair<K,V»`:

- в `map` ключ и значение концептуально разделены на `key` и `mapped value`;
- у `set<pair<...»` сортировка идёт по всей паре целиком, то есть по `first`, а при равенстве – по `second`;
- `map` предоставляет удобный доступ по ключу через `operator[]`;
- `map` моделирует ассоциативный массив, а `set<pair<...»` – просто множество пар.

10.10 Как std::map моделирует ассоциативный массив?

std::map<K, V> можно рассматривать как частично определённую дискретную функцию

$$m : K \mapsto V.$$

То есть не каждому ключу обязательно сопоставлено значение, но для существующих ключей соответствие определено.

Основные операции такие же, как у set:

- find(key) – поиск по ключу;
- insert(...) – вставка пары;
- erase(...) – удаление по ключу или итератору.

10.11 Как работает operator[] у std::map? Почему важна неконстантная перегрузка?

Операция m[k] возвращает ссылку на значение, связанное с ключом k.

Если ключ уже есть, возвращается ссылка на его значение. Если ключа нет, то в контейнер автоматически вставляется новая пара (k, V()), где V() – значение, созданное по умолчанию.

Пример

```
std::map<int, int> m;
int a = m[42];   // a == 0, a в map появится ключ 42 со значением 0
```

Примечание

В стандартном std::map оператор operator[] является только **неконстантным**. Константной перегрузки нет, потому что доступ через [] может изменить контейнер, вставив новый элемент. Для чтения из const std::map обычно используют find() или at().

Поэтому:

- для std::map<K, V> m выражение m[k] допустимо;
- для const std::map<K, V> m выражение m[k] недопустимо (ошибка компиляции).

Смысл operator[] – сделать map похожим на массив: можно писать m[k] = v. Это удобно, но нужно помнить, что при отсутствии ключа происходит вставка.

10.12 Как итерироваться по std::map? Что получается при разыменовании итератора?

Итераторы std::map тоже относятся к категории **bidirectional iterator**. То есть по ним можно двигаться вперёд и назад, но нельзя прыгать на произвольное расстояние.

При разыменовании итератора получается объект типа std::pair<const K, V>:

- first – ключ;
- second – значение.

Ключ обычно объявлен как const, чтобы нельзя было изменить порядок элементов в контейнере через итератор.

10.13 Что происходит при вставке элемента в std::map, если ключ уже существует?

Тут поведение зависит от способа вставки.

- `m[k] = v` – если ключ уже есть, то изменится только связанное значение;
- `m.insert({k, v})` – если ключ уже есть, новая пара не будет вставлена, и существующее значение не изменится.

То есть:

- `operator[]` подходит для обновления значения по ключу;
- `insert()` подходит для вставки только тогда, когда ключ отсутствует.

10.14 Как правильно искать элемент в std::map?

Идиоматический способ – использовать `find(key)`.

```
auto it = m.find(k);
if (it != m.end()) {
    // найдено: it->first == k, it->second -- значение
}
```

Такой подход полезен, потому что:

- не создаёт новый элемент;
- работает без лишней вставки;
- сразу даёт доступ к найденной паре.

Примечание

Не стоит использовать `m[k]` для проверки наличия ключа, если не нужна вставка. Иначе при отсутствии ключа в контейнере появится новый элемент с значением по умолчанию.

10.15 Чем отличаются std::multiset и std::multimap от std::set и std::map?

`std::multiset` и `std::multimap` устроены так же, как соответствующие упорядоченные контейнеры, но разрешают дубликаты ключей.

- `std::multiset` хранит несколько одинаковых значений;
- `std::multimap` хранит несколько значений под одним и тем же ключом;
- для поиска таких элементов часто используют `equal_range()`.

Примечание

Если в задаче возможны повторяющиеся ключи, нужен не `set`/`map`, а `multiset`/`multimap`.

10.16 Чем упорядоченные ассоциативные контейнеры отличаются от неупорядоченных?

Упорядоченные контейнеры хранят элементы в порядке компаратора. Неупорядоченные контейнеры используют хеш-таблицу.

- `std::unordered_set` и `std::unordered_map` не поддерживают сортированный обход;
- поиск в среднем близок к `O(1)`;
- порядок обхода зависит от хеш-таблицы и не обязан быть стабильным;
- при росте числа элементов возможен `rehash`, который перераспределяет элементы по корзинам.

Примечание

В неупорядоченных контейнерах вместо компаратора используются хеш-функция и проверка равенства ключей. Коллизии допустимы и обрабатываются внутри таблицы.

10.17 Какая типичная внутренняя реализация у `std::map` и какова асимптотика операций?

Как и `std::set`, `std::map` обычно реализован как сбалансированное бинарное дерево поиска, чаще всего красно-чёрное дерево.

Отсюда следуют асимптотики:

- поиск – `O(log n)`;
- вставка – `O(log n)`;
- удаление – `O(log n)`;
- обход всех элементов – `O(n)`.

Именно эта реализация делает `map` удобным ассоциативным массивом: быстрый поиск по ключу и упорядоченный обход по возрастанию ключей.

11 Ассоциативные контейнеры, компараторы, сравнение и сортировка

11.1 Каково назначение параметра `Compare` в шаблонах `std::set<Key, Compare>` и `std::map<Key, Value, Compare>`?

Параметр `Compare` задаёт правило упорядочивания элементов внутри ассоциативного контейнера. Именно через него контейнер понимает, какой из двух ключей «меньше», а значит – куда в дереве нужно спускаться при поиске, вставке и удалении.

Компаратор

Компаратор – это объект-функция (function object), который по двум значениям возвращает `true`, если первое должно идти раньше второго в заданном порядке.

Для `std::set<Key, Compare>` компаратор сравнивает сами элементы типа `Key`. Для `std::map<Key, Value, Compare>` компаратор сравнивает только ключи `Key`, а не пары целиком.

Примечание

В `std::map` порядок элементов определяется именно по ключу, потому что ключ однозначно задаёт место элемента в дереве.

11.2 Какая типичная реализация у `std::set` и `std::map`? Почему при поиске используется сравнение?

Типичная реализация ассоциативных контейнеров – это **сбалансированное бинарное дерево поиска** (обычно красно-чёрное дерево). Поэтому для определения пути к элементу нужно сравнивать ключи: идти в левое или правое поддерево.

При использовании компаратора по умолчанию таким сравнением является операция “меньше”, то есть `operator<`. Именно поэтому ключи должны быть *comparable*: их можно упорядочить некоторым строгим слабым порядком (*strict weak ordering*).

Отсюда следуют типичные асимптотики:

$$\text{find, insert, erase} = O(\log n).$$

11.3 Какие стандартные компараторы используются по умолчанию?

По умолчанию в упорядоченных ассоциативных контейнерах используется `std::less<T>`.

`std::less<T>`

`std::less<T>` – компаратор, который реализует сравнение вида `a < b`.

Аналогично, `std::greater<T>` задаёт обратный порядок, то есть сравнение вида `a > b`.

- `std::set<int>` по умолчанию отсортирован по возрастанию.
- `std::set<int, std::greater<int»` – по убыванию.

При использовании `std::less<T>` тип `T` должен быть сравним через `operator<`. Если такого оператора нет, то нужно либо определить его, либо передать пользовательский компаратор.

11.4 Как задаётся пользовательский компаратор?

Пользовательский компаратор можно задать несколькими способами:

- через структуру или класс с перегруженным `operator()`;
- через лямбда-выражение, если оно подходит по типу;
- через готовый функциональный объект из стандартной библиотеки.

Пример

```
struct ByAbs {
    bool operator()(int a, int b) const {
        return std::abs(a) < std::abs(b);
    }
};

std::set<int, ByAbs> s;
```

Здесь множество упорядочивается не по обычному значению, а по модулю.

Примечание

Компаратор должен задавать корректный строгий слабый порядок. Если это свойство нарушить, контейнер может работать некорректно.

11.5 Каковы способы реализации пользовательского компаратора?

На практике чаще всего используют такие формы:

1. **Структура-функтор**:

   ```
   struct Cmp {
       bool operator()(const T& a, const T& b) const {
           ...
       }
   };
   ```

2. **Лямбда**, если компаратор нужен локально:

   ```
   auto cmp = [](const T& a, const T& b) {
       return ...;
   };
   ```

3. **std::function**, если нужен полиморфизм, но обычно это избыточно и медленнее.

Для ассоциативных контейнеров особенно удобен функтор, потому что тип компаратора должен быть известен на этапе компиляции.

11.6 Чем пользовательский компаратор отличается от перегрузки `operator<`?

Оба подхода решают задачу упорядочивания, но делают это по-разному.

- **Перегрузка** `operator<` задаёт естественный порядок самого типа.
- **Пользовательский компаратор** задаёт порядок только для конкретного контейнера или алгоритма.

Преимущества пользовательского компаратора:

- можно хранить один и тот же тип в разных контейнерах с разными порядками;
- не нужно навязывать всем пользователям класса единственный вариант сравнения;
- удобно для случаев, когда порядок зависит от задачи, а не от природы типа.

Пример

Один и тот же тип `Person` можно отсортировать:

- по фамилии в одном контейнере;
- по возрасту в другом;
- по id в третьем.

Если бы порядок задавался только через `operator<`, это было бы менее гибко.

11.7 Как устроена «автоматическая сортировка» ассоциативных контейнеров?

Упорядоченные ассоциативные контейнеры сами поддерживают порядок при каждой вставке. Это не отдельная операция сортировки, а свойство внутренней структуры данных.

То есть:

- при `insert` элемент помещается в нужную позицию;
- при `erase` дерево перестраивается;
- при `find` путь ищется по сравнению ключей;
- при обходе контейнера элементы уже идут в отсортированном порядке.

Примечание

У `std::set` и `std::map` нельзя сказать: «отсортируй контейнер ещё раз». Они уже всегда хранят элементы в отсортированном виде.

11.8 Как «сортируют» элементы непустого `std::set`?

Формально `std::set` не сортируют, потому что он уже упорядочен. Если нужен другой порядок, обычно делают одно из двух:

1. создают новый `set` с другим компаратором;
2. копируют элементы в `std::vector` и сортируют уже его.

Пример

```
std::set<int> s = {3, 1, 4};
std::set<int, std::greater<int>> t(s.begin(), s.end());
```

Это даёт тот же набор элементов, но уже с другим порядком обхода.

11.9 Как «сортируют» элементы непустого std::vector?

Для `std::vector` сортировка выполняется алгоритмом `std::sort`. Это уже *ручная* сортировка линейного контейнера, в отличие от автоматической упорядоченности `set` и `map`.

Пример

```
std::vector<int> v = {3, 1, 4, 2};
std::sort(v.begin(), v.end());
```

Алгоритм `std::sort` переставляет элементы контейнера так, чтобы они шли в порядке, заданном компаратором.

11.10 Какие требования накладываются на линейный контейнер, чтобы его можно было сортировать std::sort?

Для `std::sort` нужны:

- **random access iterators**, потому что алгоритм активно работает с позицией элементов;
- возможность перестановки элементов;
- корректный компаратор, задающий строгий слабый порядок.

Поэтому `std::vector` можно сортировать, а `std::list` – нельзя, потому что у неё не random access iterators.

Примечание

Сортировка `std::sort` не требует непрерывной памяти как обязательного свойства, но для `std::vector` она обычно есть, и это делает контейнер особенно удобным.

11.11 Можно ли отсортировать поддиапазон std::vector?

Да. Алгоритм `std::sort` работает не только со всем контейнером, но и с любым поддиапазоном, заданным итераторами.

Пример

```
std::sort(v.begin() + l, v.begin() + r);
```

Здесь сортируется только часть вектора на полуинтервале `[l, r)`.
Это удобно, если нужно упорядочить не весь контейнер, а только его фрагмент.

11.12 Можно ли упорядочить только часть std::set или std::map?

Нет, в обычном смысле – нельзя. Ассоциативный контейнер всегда хранит **весь** набор элементов в соответствии с одним общим порядком, заданным компаратором.

Нельзя выбрать произвольное подмножество элементов и «отсортировать только его» внутри set или map, потому что:

- контейнер уже сам поддерживает порядок;
- элементы связаны с внутренним деревом;
- ключи в set и map фактически недоступны для произвольного переупорядочивания.

Если нужен другой порядок только для части данных, обычно:

1. копируют нужные элементы в vector;
2. сортируют vector нужным образом;
3. либо используют отдельный контейнер с другим компаратором.

11.13 Можно ли использовать один и тот же компаратор для std::set, std::map и std::vector?

Да, но только если он подходит по типам сравниваемых объектов.

- Для std::set<Key, Compare> и std::map<Key, Value, Compare> компаратор сравнивает ключи типа Key.
- Для std::vector<T> компаратор сравнивает элементы типа T.

То есть один и тот же *тип* компаратора можно использовать, если он умеет сравнивать нужные значения. Например, один функтор может сравнивать и ключи в set/map, и элементы vector, если все они имеют один и тот же тип и один и тот же смысл порядка.

11.14 Что такое неупорядоченные ассоциативные контейнеры? Как они устроены?

Неупорядоченные ассоциативные контейнеры — это std::unordered_set, std::unordered_map и их многократные версии. Они организованы как хеш-таблицы, а не как деревья.

- для доступа используется хеш-функция;
- элементы размещаются по корзинам (buckets);
- коллизии разрешаются внутри таблицы;
- порядок обхода не связан с порядком ключей.

Примечание

В неупорядоченных контейнерах вместо компаратора нужен хешер и проверка равенства ключей. Это даёт быстрый поиск в среднем, но не упорядоченный обход.

11.15 Когда неупорядоченные контейнеры полезнее упорядоченных?

Неупорядоченные контейнеры полезны тогда, когда нужен быстрый доступ по ключу, а сортированный обход не важен.

- если важен порядок ключей и диапазонные операции, выбирают `set`/`map`;
- если нужен только быстрый поиск, часто удобнее `unordered_set`/`unordered_map`;
- если нужен и порядок, и быстрый поиск, приходится выбирать по приоритетам задачи.